Projector-Generated Special Functions V

Log-Gamma Gabor Packets, Gamma–Sech Ambiguity Kernels, Meixner–Pollaczek Spectra, and SU(1, 1) Duality

Komorebi P.

Abstract

This paper identifies the universal phase-space object hidden in the large-depth limit of the squarefree packet kernels developed in the preceding two papers. Let

$$\alpha = \frac{2a-1}{\beta} > 0.$$

After logarithmic centering, the normalized arithmetic depth-time kernel converges to

$$\boxed{\mathfrak{A}_\alpha(h,\eta) = \frac{\Gamma(\alpha+\mathrm{i}\eta)}{\Gamma(\alpha)}\operatorname{sech}^{\alpha+\mathrm{i}\eta}\left(\frac{h}{2}\right).}$$

The limit is not only a closed asymptotic formula. It is exactly the centered Gabor ambiguity function of the normalized log-Gamma packet

$$g_\alpha(y) = \Gamma(\alpha)^{-1/2}\exp\left(\frac{\alpha y}{2} - \frac{\mathrm{e}^y}{2}\right).$$

This identification generates a complete special-function system. The ambiguity function is zero-free on real phase space, obeys an exact shape shift in α, and solves a Pöschl–Teller equation. The Fourier transform of g_α is a Gamma function. Its squared modulus is the symmetric Meixner density and its characteristic function is the exact depth-overlap law $\operatorname{sech}^\alpha(h/2)$. The Wigner distribution has a Kontorovich–Lebedev closed form involving $K_{2\mathrm{i}\omega}(\mathrm{e}^y)$.

The packet has exact position and momentum variances

$$\operatorname{Var} Y = \psi_1(\alpha), \qquad \operatorname{Var} P = \frac{\alpha}{4}.$$

Its Heisenberg product is strictly larger than $1/2$ at every finite α and tends to $1/2$ as α grows. The Weyl orbit has constant quantum metric and unit Berry curvature in the selected coordinates.

The orthogonal polynomial structure is also exact. Laguerre packets on logarithmic position space transform into the same Gamma envelope multiplied by orthonormal Meixner–Pollaczek polynomials. Both bases carry one positive discrete-series representation of $\mathfrak{su}(1,1)$. The Laguerre multiplication operator and the Meixner frequency operator are two linear combinations of the same ladder generators.

Finally, the discrete arithmetic depth is extended to a continuous parameter. The resulting fractional-depth Möbius transfer admits an infinite binomial expansion in reciprocal L-values and an exact generator relation across the parameter a. The unconditional positive and representation-theoretic results do not prove the remaining principal Möbius coherence bound and do not prove RH. They isolate a universal phase-space geometry shared by the positive packet family.

All figures were generated by the accompanying Python program. The release contains 26 vector figures, matching PNG files, exact tables, 19 built-in checks, 19 independent tests, and the complete Python source.

Status and claim boundary

The paper proves the log-Gamma ambiguity representation, the Gamma–sech closed form, its zero-free real phase-space law, the parameter shift, the Pöschl–Teller equation, the Fourier and Meixner density formulas, the Wigner–Kontorovich–Lebedev identity, the exact uncertainty formulas, the constant orbit geometry, the Laguerre–Meixner Fourier duality, and the $\mathfrak{su}(1,1)$ operator realization. It also proves the continuous-depth binomial expansion and the fractional generator relation under the stated absolute-convergence hypotheses.

The squarefree arithmetic convergence theorem uses the standard squarefree counting asymptotic and is uniform only on compact sets of the ratio and time parameters described below. The associated numerical figures are finite checks of that theorem, not substitutes for it.

The classical inputs include Gamma and polygamma functions, Laguerre polynomials, Meixner–Pollaczek polynomials, Gabor ambiguity functions, Wigner distributions, the Kontorovich–Lebedev kernel, and the positive discrete series of $\mathrm{SU}(1,1)$. The generated contribution is the exact bridge showing that these objects arise from one arithmetic packet limit with one common normalization and one common ladder system.

The logarithmic quantum coordinate is compatible with the external 2026 construction based on the number-basis Hamiltonian $H\,|n\rangle = \log n\,|n\rangle$. The present observable is a scale-filtered squarefree kernel and is not the direct zeta amplitude of that work.

None of the unconditional results proves RH. The open part remains the signed time-zero coherence of the principal Möbius packet.

Contents

Status and claim boundary i

I The arithmetic limit and its exact packet window 1

1 The squarefree depth-time kernel 1

2 The log-Gamma packet 3

3 The exact Gamma–sech ambiguity function 5

II Fourier, Wigner, and information-geometric coordinates 8

4 The Meixner spectral law 8

5 The Wigner–Kontorovich–Lebedev function 10

6 Uncertainty and quantum geometry 12

III Orthogonal packet bases and one ladder algebra 14

7 Laguerre packets on logarithmic position space 14

8 Meixner–Pollaczek packets in Mellin frequency 15

9 The common $\mathfrak{su}(1,1)$ ladder 17

IV Fractional depth, quantum realization, and the RH boundary 20

10 Continuous packet depth 20

11 Finite arithmetic convergence 22

12 Quantum and signal realization **24**

13 The remaining RH boundary **25**

V Verification, formula registry, and source **26**

14 Verification ledger **27**

15 Formula registry **28**

16 Relation to established function theory **28**

A Complete Python implementation **30**

B Reproduction **45**

List of Figures

1.1 The generated theory. The positive squarefree depth-time kernel converges to a log-Gamma Gabor packet. Exact transforms of that packet generate the ambiguity function, spectral density, Wigner function, orthogonal polynomials, and ladder operators studied in the rest of the paper. 2

2.1 Normalized log-Gamma windows. Increasing α moves the packet to larger log scale and makes the local profile increasingly Gaussian after centering. 4

2.2 Position probability densities. The squared packet is the logarithm of a Gamma random variable. Its exact mean and variance are the digamma and trigamma functions. 4

3.1 Magnitude of the ambiguity function. The generated packet has no real phase-space zeros. Decay in depth displacement comes from the hyperbolic secant. Decay in modulation comes from the Gamma factor. 6

3.2 Phase of the ambiguity function. The phase separates into a Gamma phase and a depth-dependent carrier produced by the complex power of the hyperbolic secant. 6

3.3 One-dimensional ambiguity slices. The left panel varies the depth displacement at fixed Mellin frequencies. The right panel varies the Mellin-frequency shift at fixed depth displacements. 7

3.4 Pöschl–Teller realization. The left panel shows the complex potential coefficient. The right panel verifies the differential equation numerically at high resolution. 8

4.1 Symmetric Meixner spectral densities. The logarithmic packet has a Gamma-function Fourier envelope. Its squared modulus is the orthogonality density used by the Meixner–Pollaczek packet basis. 9

4.2 Exact depth autocorrelation. The universal cross-depth overlap is the characteristic function of the Meixner density and the zero-modulation slice of the ambiguity function. . 9

5.1 The Wigner–Kontorovich–Lebedev packet. The distribution is an exact Bessel function of imaginary order. Positive and negative regions encode the nonclassical phase-space interference of the packet. 11

5.2 Wigner marginals. Numerical integration of the Bessel closed form reproduces both the exact log-Gamma position density and the exact Meixner spectral density. 11

6.1 Exact uncertainty product. The packet is not a Gaussian at finite shape, but after centering and large-α scaling it approaches the minimum-uncertainty regime. 13

6.2 Quantum geometry of the Weyl orbit. Translation and modulation inherit exact metric coefficients from the packet variances. The canonical commutator supplies the constant symplectic curvature. 14

7.1 Laguerre packets in log coordinates. The packets form an exact orthonormal multioscillation family around the universal log-Gamma envelope. 15

8.1 Meixner–Pollaczek packets. Multiplying each orthonormal polynomial by the square root of its Gamma density gives the frequency-side packet family. 16

8.2 Exact Fourier duality. The real and imaginary parts alternate with degree through the phase i^n. The Gamma envelope is common to every packet. 17

9.1 Laguerre Jacobi matrix. The position-side multiplication operator is tridiagonal in the logarithmic Laguerre packet basis. 18

9.2 Meixner Jacobi matrix. The Mellin-frequency operator is another tridiagonal combination of the same ladder generators. 19

9.3 One ladder and two atlases. The Laguerre and Meixner operators are not unrelated recurrences. They are two projections of the same positive discrete-series ladder. 20

10.1 Continuous depth. The integer Bernoulli finite-difference orbit is embedded into a smooth depth evolution generated by an infinite parameter-shift operator. 21

10.2 Fractional binomial reconstruction. The infinite reciprocal-zeta expansion converges to the direct Möbius packet calculation for the noninteger depth $r = 5/2$. 22

11.1 Arithmetic convergence. The finite squarefree packet moves toward the universal Gamma–sech kernel as the active scale grows. The displayed reference line is only a visual slope guide. 23

11.2 Complex trajectory comparison. After centering, the finite arithmetic correlation and the universal phase-space curve nearly coincide over the tested time interval. 23

12.1 Packet lattice. Circle area is proportional to the exact overlap magnitude. The generated function supplies a natural nonuniform coherent-state geometry in log depth and Mellin frequency. 24

12.2 Quantum and signal path. The logarithmic Hamiltonian supplies the time phase. The depth filter supplies scale selection. Their centered overlap converges to the universal log-Gamma packet geometry. 25

13.1 The generated family. The Gamma–sech ambiguity function, log-Gamma window, Meixner spectrum, Wigner–Bessel function, Laguerre–Meixner Fourier pair, ladder algebra, and fractional-depth transfer are exact coordinates of one universal packet system. 26

Part I

The arithmetic limit and its exact packet window

Chapter 1

The squarefree depth-time kernel

Fix

$$a > \frac{1}{2}, \qquad \beta > 0, \qquad \alpha = \frac{2a-1}{\beta}.$$

The positive depth-time kernel is

$$\boxed{\mathcal{K}_{a,\beta}(k,\ell,\tau) = \sum_{n\geq 1} \frac{\mu(n)^2}{n^{2a+\mathrm{i}\tau}} \left(1 - \frac{1}{n^\beta}\right)^{k+\ell}.}$$

When a character conductor is present, one adds the coprimality restriction or the factor $|\chi(n)|^2$. The principal formulas below are unchanged apart from the squarefree density constant.

Set

$$m = k + \ell, \qquad \eta = \frac{\tau}{\beta}.$$

The active integers satisfy $n^\beta \asymp m$. This is the depth localization found in Paper III. Paper IV derived the separate time and cross-depth limits. The first result here joins them.

Theorem 1.1 (Joint arithmetic phase-space limit)**.** *Assume that $k, \ell \to \infty$ while k/ℓ remains in a compact subset of $(0,\infty)$ and τ remains in a compact real interval. Let*

$$h = \log \frac{k}{\ell}.$$

Then

$$\boxed{\mathrm{e}^{\mathrm{i}\eta \log(2\sqrt{k\ell})} \frac{\mathcal{K}_{a,\beta}(k,\ell,\tau)}{\sqrt{\mathcal{K}_{a,\beta}(k,k,0)\mathcal{K}_{a,\beta}(\ell,\ell,0)}} \longrightarrow \mathfrak{A}_\alpha(h,\eta),}$$

where

$$\boxed{\mathfrak{A}_\alpha(h,\eta) = \frac{\Gamma(\alpha+\mathrm{i}\eta)}{\Gamma(\alpha)} \operatorname{sech}^{\alpha+\mathrm{i}\eta}\left(\frac{h}{2}\right).}$$

Proof. Let

$$A(x) = \sum_{n\leq x} \mu(n)^2.$$

The squarefree counting asymptotic is

$$A(x) = \delta x + O(x^{1/2}), \qquad \delta = \frac{1}{\zeta(2)}.$$

Partial summation converts the kernel into a Stieltjes integral against A. On replacing $A(x)$ by its main term and scaling

$$x = m^{1/\beta} u,$$

one obtains

$$\mathcal{K}_{a,\beta}(k,\ell,\tau) \sim \frac{\delta}{\beta}\Gamma(\alpha+\mathrm{i}\eta)m^{-\alpha-\mathrm{i}\eta}.$$

The same calculation at $(k,k,0)$ and $(\ell,\ell,0)$ gives

$$\mathcal{K}_{a,\beta}(k,k,0) \sim \frac{\delta}{\beta}\Gamma(\alpha)(2k)^{-\alpha}$$

and the analogous formula for ℓ. Therefore the normalized magnitude factor is

$$\left(\frac{2\sqrt{k\ell}}{k+\ell}\right)^{\alpha} = \operatorname{sech}^{\alpha}\left(\frac{h}{2}\right).$$

The carrier phase leaves

$$\left(\frac{2\sqrt{k\ell}}{k+\ell}\right)^{\mathrm{i}\eta} = \operatorname{sech}^{\mathrm{i}\eta}\left(\frac{h}{2}\right).$$

Combining these terms proves the formula. Uniformity on compact parameter sets follows from the same partial-summation estimate because the scaled kernels and their first derivatives remain integrably dominated there. □

From arithmetic depth packets to a universal phase-space function

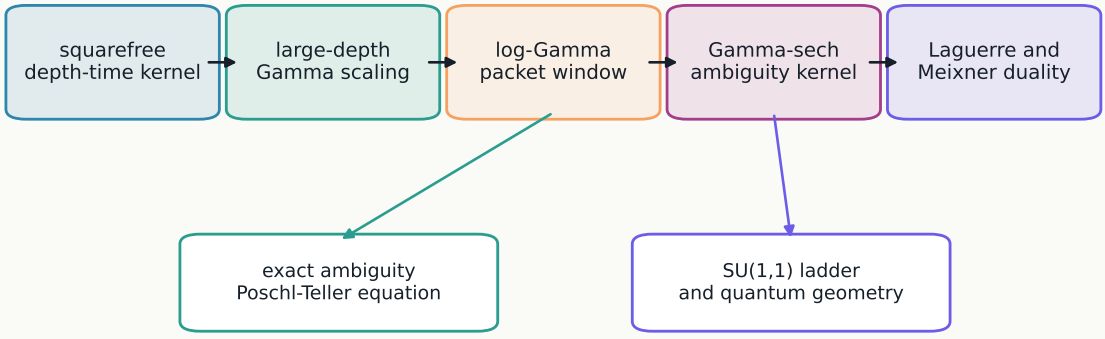


Figure 1.1: The generated theory. The positive squarefree depth-time kernel converges to a log-Gamma Gabor packet. Exact transforms of that packet generate the ambiguity function, spectral density, Wigner function, orthogonal polynomials, and ladder operators studied in the rest of the paper.

Worked example 1.2 (Principal parameters)**.** For

$$a = 2, \qquad \beta = 2,$$

one has

$$\alpha = \frac{3}{2}.$$

Taking $\ell = 4k$ gives

$$h = -\log 4.$$

At $\tau = 2.4$, one has $\eta = 1.2$. The exact limiting value is

$$\mathfrak{A}_{3/2}(-\log 4, 1.2).$$

The finite squarefree kernels used in figures 11.1 and 11.2 converge to this complex value after the carrier phase is removed.

Chapter 2

The log-Gamma packet

Definition 2.1 (Normalized log-Gamma window)**.** For $\alpha > 0$, define

$$\boxed{g_\alpha(y) = \frac{1}{\sqrt{\Gamma(\alpha)}} \exp\!\left(\frac{\alpha y}{2} - \frac{\mathrm{e}^y}{2}\right), \qquad y \in \mathbb{R}.}$$

Its squared modulus is

$$\boxed{|g_\alpha(y)|^2 = \frac{1}{\Gamma(\alpha)} \exp(\alpha y - \mathrm{e}^y).}$$

Under the change of variables

$$u = \mathrm{e}^y,$$

this is the Gamma$(\alpha, 1)$ law in logarithmic coordinates.

Proposition 2.2 (Normalization)**.** *For every $\alpha > 0$,*

$$\|g_\alpha\|_{L^2(\mathbb{R})} = 1.$$

Proof. The substitution $u = \mathrm{e}^y$ gives

$$\int_{-\infty}^{\infty} |g_\alpha(y)|^2 \,\mathrm{d}y = \frac{1}{\Gamma(\alpha)} \int_0^\infty u^{\alpha-1}\mathrm{e}^{-u} \,\mathrm{d}u = 1.$$

□

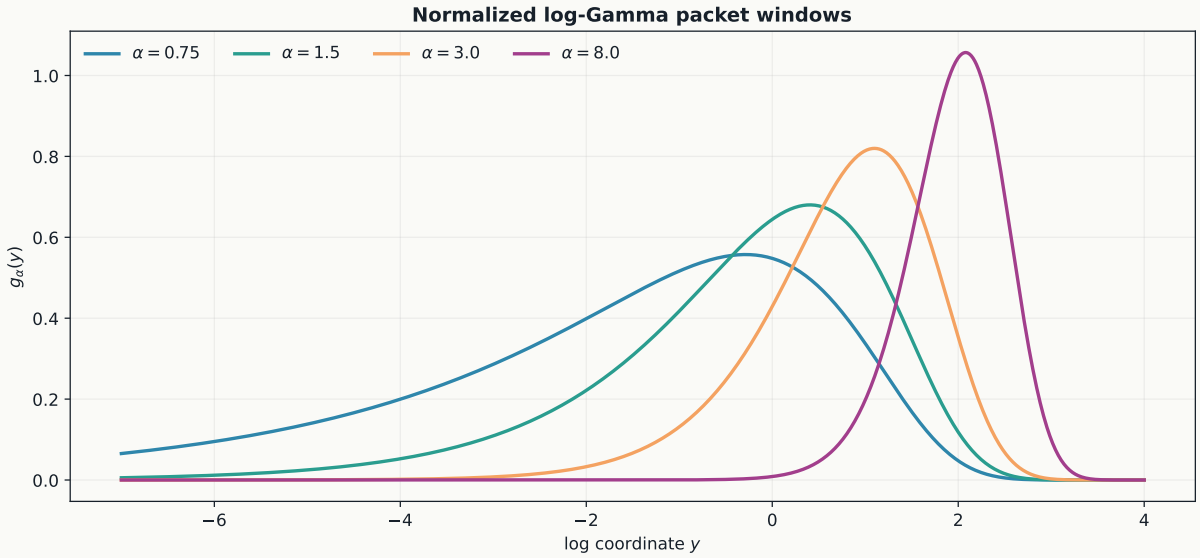


Figure 2.1: Normalized log-Gamma windows. Increasing α moves the packet to larger log scale and makes the local profile increasingly Gaussian after centering.

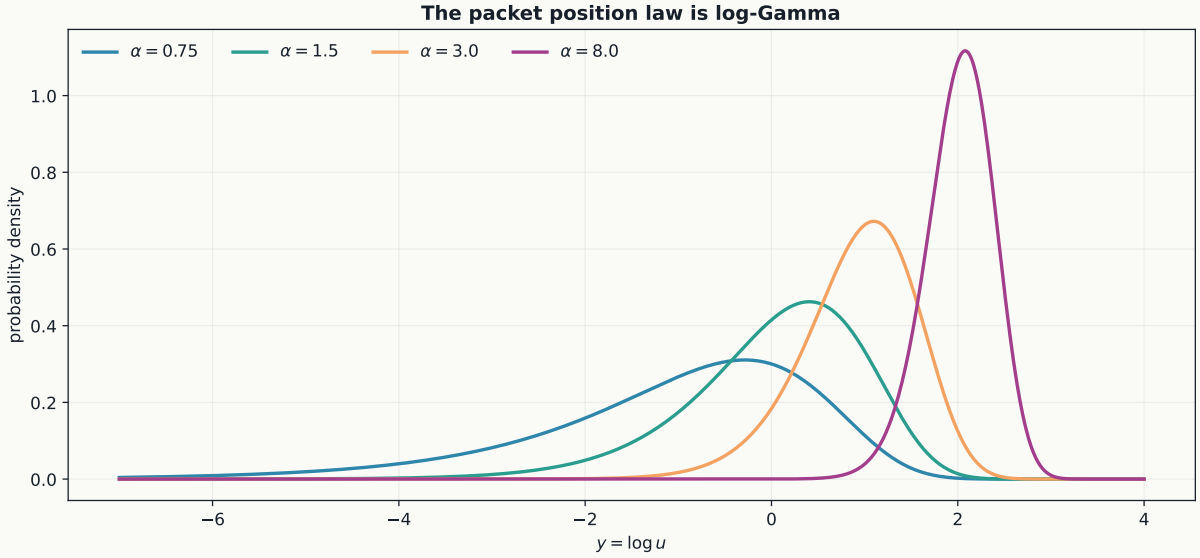


Figure 2.2: Position probability densities. The squared packet is the logarithm of a Gamma random variable. Its exact mean and variance are the digamma and trigamma functions.

Chapter 3

The exact Gamma–sech ambiguity function

Use the centered ambiguity convention

$$\mathcal{A}_g(h,\eta) = \int_{\mathbb{R}} g\left(y + \frac{h}{2}\right) \overline{g\left(y - \frac{h}{2}\right)} \mathrm{e}^{\mathrm{i}\eta y}\, \mathrm{d}y.$$

The packet is real, so the conjugate may be omitted.

Theorem 3.1 (Exact ambiguity formula)**.** *For every $\alpha > 0$ and real h, η,*

$$\boxed{\mathcal{A}_{g_\alpha}(h,\eta) = \mathfrak{A}_\alpha(h,\eta) = \frac{\Gamma(\alpha + \mathrm{i}\eta)}{\Gamma(\alpha)} \operatorname{sech}^{\alpha+\mathrm{i}\eta}\left(\frac{h}{2}\right).}$$

Proof. The product of the two shifted windows is

$$\frac{1}{\Gamma(\alpha)} \exp\left(\alpha y - \mathrm{e}^y \cosh\frac{h}{2}\right).$$

Therefore

$$\mathcal{A}_{g_\alpha}(h,\eta) = \frac{1}{\Gamma(\alpha)} \int_{\mathbb{R}} \exp\left((\alpha + \mathrm{i}\eta)y - \mathrm{e}^y \cosh\frac{h}{2}\right) \mathrm{d}y.$$

Set

$$u = \mathrm{e}^y \cosh\frac{h}{2}.$$

The integral becomes

$$\frac{\Gamma(\alpha + \mathrm{i}\eta)}{\Gamma(\alpha)} \cosh^{-\alpha-\mathrm{i}\eta}\left(\frac{h}{2}\right).$$

□

Corollary 3.2 (Real phase-space nonvanishing)**.** *For every real h and η,*

$$\boxed{\mathfrak{A}_\alpha(h,\eta) \neq 0.}$$

Proof. The Gamma function has no zeros. The hyperbolic secant is strictly positive on the real line. □

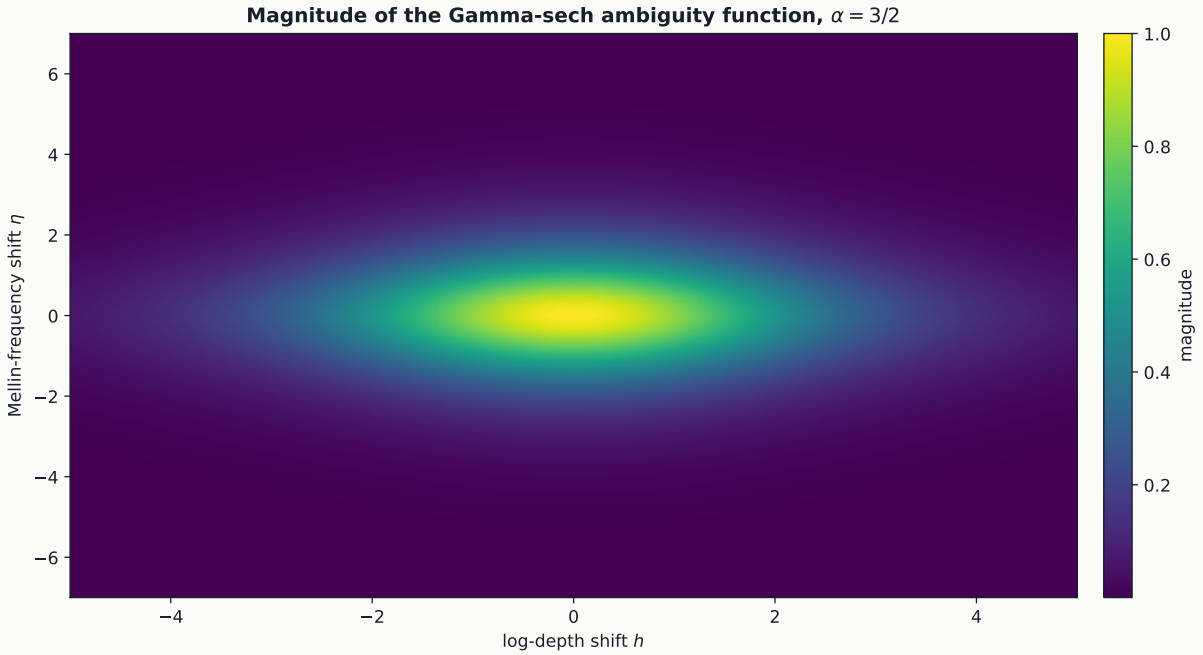


Figure 3.1: Magnitude of the ambiguity function. The generated packet has no real phase-space zeros. Decay in depth displacement comes from the hyperbolic secant. Decay in modulation comes from the Gamma factor.

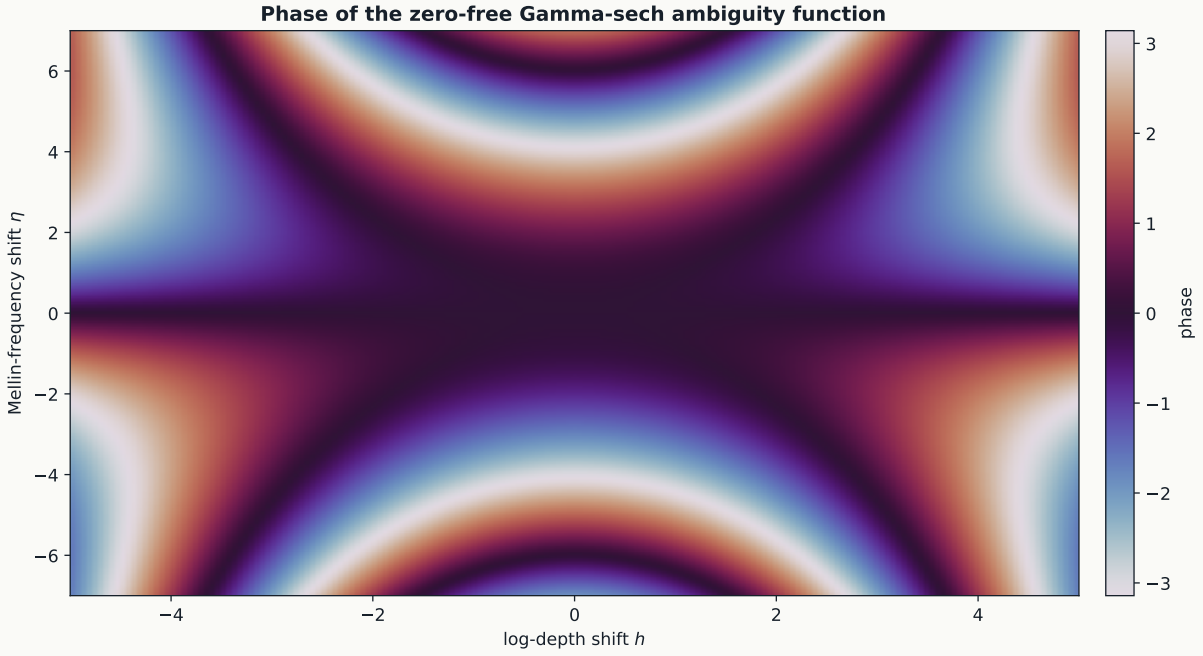


Figure 3.2: Phase of the ambiguity function. The phase separates into a Gamma phase and a depth-dependent carrier produced by the complex power of the hyperbolic secant.

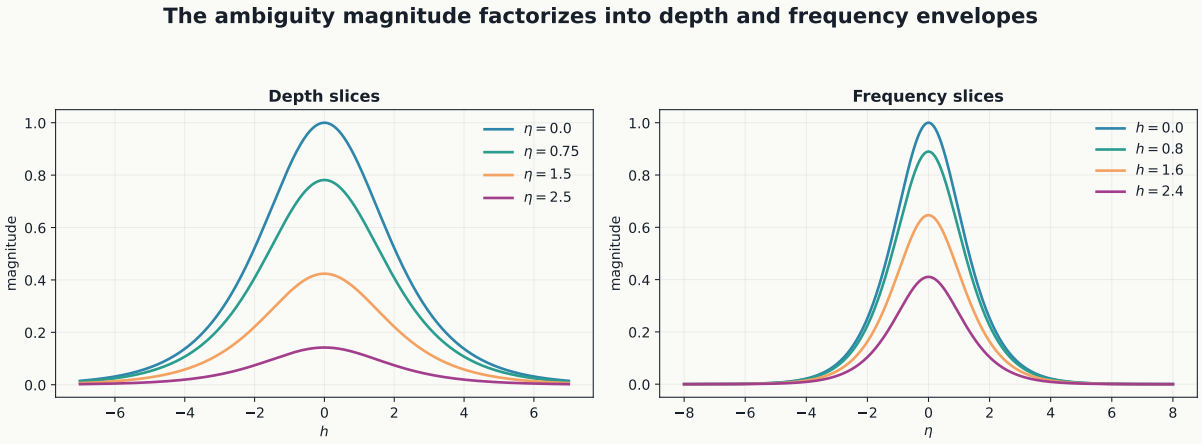


Figure 3.3: One-dimensional ambiguity slices. The left panel varies the depth displacement at fixed Mellin frequencies. The right panel varies the Mellin-frequency shift at fixed depth displacements.

Proposition 3.3 (Shape shift)**.** *The family satisfies*

$$\boxed{\mathfrak{A}_{\alpha+1}(h,\eta) = \frac{\alpha+\mathrm{i}\eta}{\alpha}\,\operatorname{sech}\!\left(\frac{h}{2}\right)\mathfrak{A}_{\alpha}(h,\eta).}$$

Proof. Use

$$\Gamma(\alpha+1+\mathrm{i}\eta) = (\alpha+\mathrm{i}\eta)\Gamma(\alpha+\mathrm{i}\eta)$$

and

$$\Gamma(\alpha+1) = \alpha\Gamma(\alpha).$$

□

Theorem 3.4 (Pöschl–Teller equation)**.** *Let*

$$\nu = \alpha + \mathrm{i}\eta.$$

For fixed η, the function $h \mapsto \mathfrak{A}_{\alpha}(h,\eta)$ satisfies

$$\boxed{\left[-4\frac{\mathrm{d}^2}{\mathrm{d}h^2} - \nu(\nu+1)\operatorname{sech}^2\!\left(\frac{h}{2}\right)\right]\mathfrak{A}_{\alpha}(h,\eta) = -\nu^2\mathfrak{A}_{\alpha}(h,\eta).}$$

Proof. The Gamma prefactor is independent of h. Direct differentiation of

$$f(h) = \operatorname{sech}^{\nu}(h/2)$$

gives

$$4f''(h) = \left[\nu^2 - \nu(\nu+1)\operatorname{sech}^2(h/2)\right] f(h).$$

Rearranging proves the equation. □

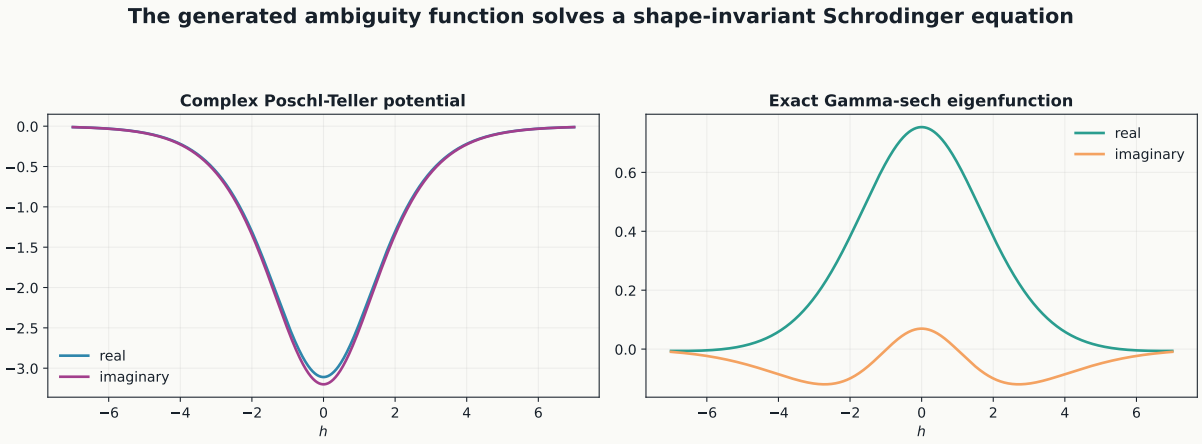


Figure 3.4: Pöschl–Teller realization. The left panel shows the complex potential coefficient. The right panel verifies the differential equation numerically at high resolution.

Part II

Fourier, Wigner, and information-geometric coordinates

Chapter 4

The Meixner spectral law

Use the unitary Fourier convention

$$\widehat{f}(\omega) = \frac{1}{\sqrt{2\pi}} \int_{\mathbb{R}} f(y) \mathrm{e}^{-\mathrm{i}\omega y} \, \mathrm{d}y.$$

Theorem 4.1 (Fourier transform of the log-Gamma packet)**.** *For real* ω*,*

$$\boxed{\widehat{g}_\alpha(\omega) = \frac{2^{\alpha/2-\mathrm{i}\omega}\Gamma(\alpha/2 - \mathrm{i}\omega)}{\sqrt{2\pi\Gamma(\alpha)}}.}$$

Proof. Set $x = \mathrm{e}^y$. Then

$$\widehat{g}_\alpha(\omega) = \frac{1}{\sqrt{2\pi\Gamma(\alpha)}} \int_0^\infty x^{\alpha/2-\mathrm{i}\omega-1} \mathrm{e}^{-x/2} \, \mathrm{d}x.$$

The Gamma integral gives the result. □

The spectral probability density is

$$\boxed{\rho_\alpha(\omega) = |\widehat{g}_\alpha(\omega)|^2 = \frac{2^{\alpha-1}}{\pi\Gamma(\alpha)} \left|\Gamma\left(\frac{\alpha}{2} + \mathrm{i}\omega\right)\right|^2.}$$

Corollary 4.2 (Exact characteristic function)**.**

$$\boxed{\int_{\mathbb{R}} \rho_\alpha(\omega) \mathrm{e}^{\mathrm{i}\omega h} \, \mathrm{d}\omega = \operatorname{sech}^\alpha\left(\frac{h}{2}\right).}$$

Proof. The left side is the Fourier autocorrelation of g_α. The ambiguity theorem at $\eta = 0$ gives the right side. □

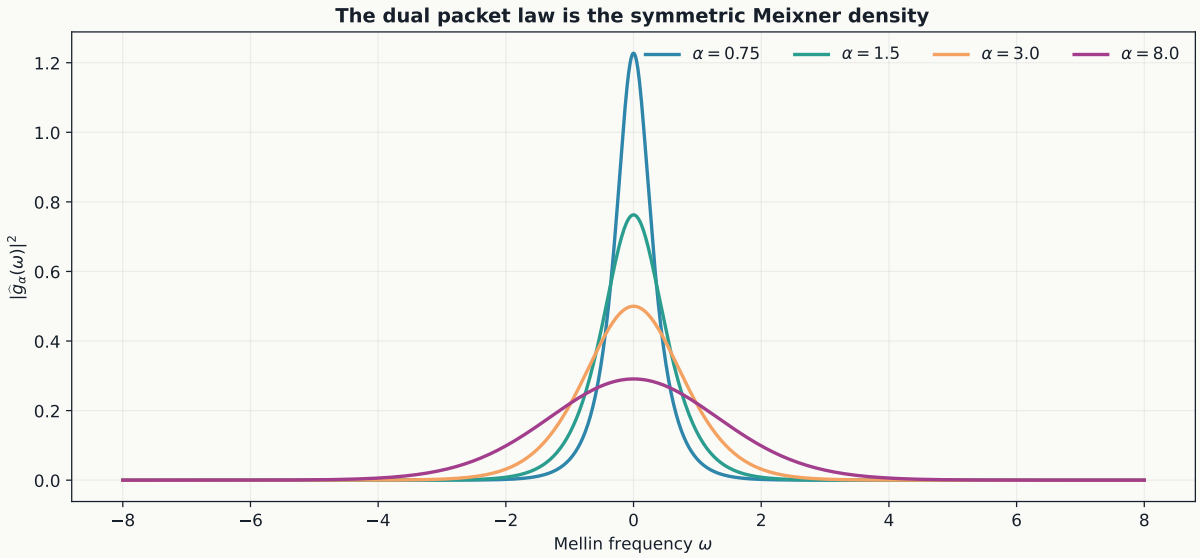


Figure 4.1: Symmetric Meixner spectral densities. The logarithmic packet has a Gamma-function Fourier envelope. Its squared modulus is the orthogonality density used by the Meixner–Pollaczek packet basis.

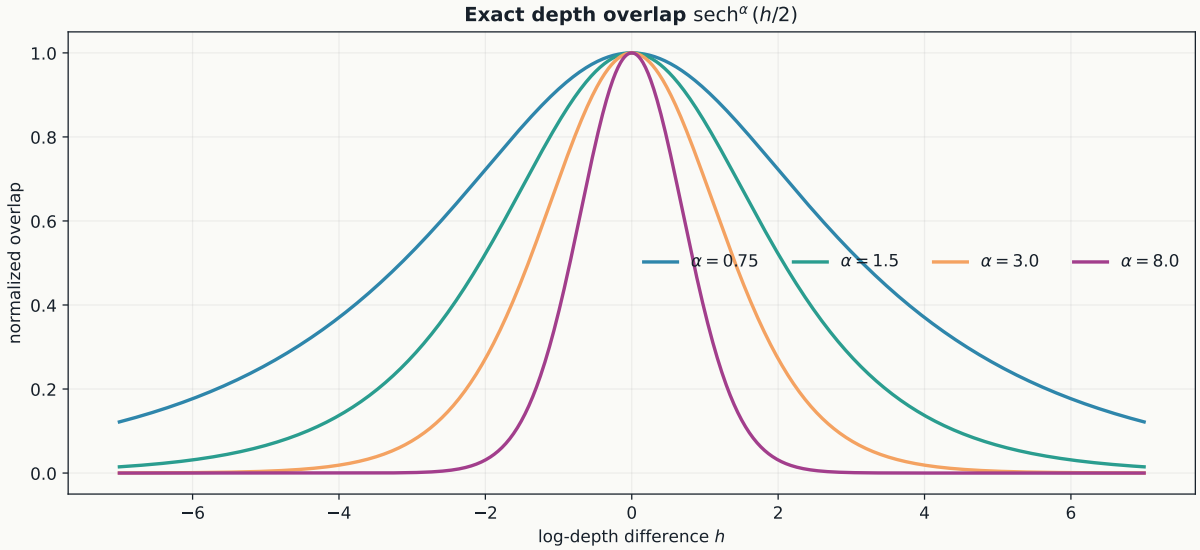


Figure 4.2: Exact depth autocorrelation. The universal cross-depth overlap is the characteristic function of the Meixner density and the zero-modulation slice of the ambiguity function.

Chapter 5

The Wigner–Kontorovich–Lebedev function

Define the Wigner distribution by

$$W_\alpha(y,\omega) = \frac{1}{2\pi}\int_{\mathbb{R}} g_\alpha\left(y+\frac{h}{2}\right) g_\alpha\left(y-\frac{h}{2}\right) \mathrm{e}^{-\mathrm{i}\omega h}\,\mathrm{d}h.$$

Theorem 5.1 (Closed Wigner formula)**.**

$$\boxed{W_\alpha(y,\omega) = \frac{2\mathrm{e}^{\alpha y}}{\pi\Gamma(\alpha)} K_{2\mathrm{i}\omega}(\mathrm{e}^y).}$$

Proof. The product of the shifted windows equals

$$\Gamma(\alpha)^{-1}\exp(\alpha y - \mathrm{e}^y\cosh(h/2))\,.$$

Set $h = 2t$. The remaining integral is

$$2\int_{-\infty}^{\infty} \mathrm{e}^{-\mathrm{e}^y\cosh t}\mathrm{e}^{-2\mathrm{i}\omega t}\,\mathrm{d}t.$$

The integral representation of the modified Bessel function gives

$$\int_{-\infty}^{\infty} \mathrm{e}^{-x\cosh t}\mathrm{e}^{-2\mathrm{i}\omega t}\,\mathrm{d}t = 2K_{2\mathrm{i}\omega}(x).$$

Substitution yields the formula. □

Corollary 5.2 (Exact marginals)**.**

$$\boxed{\int_{\mathbb{R}} W_\alpha(y,\omega)\,\mathrm{d}\omega = |g_\alpha(y)|^2}$$

and

$$\boxed{\int_{\mathbb{R}} W_\alpha(y,\omega)\,\mathrm{d}y = \rho_\alpha(\omega).}$$

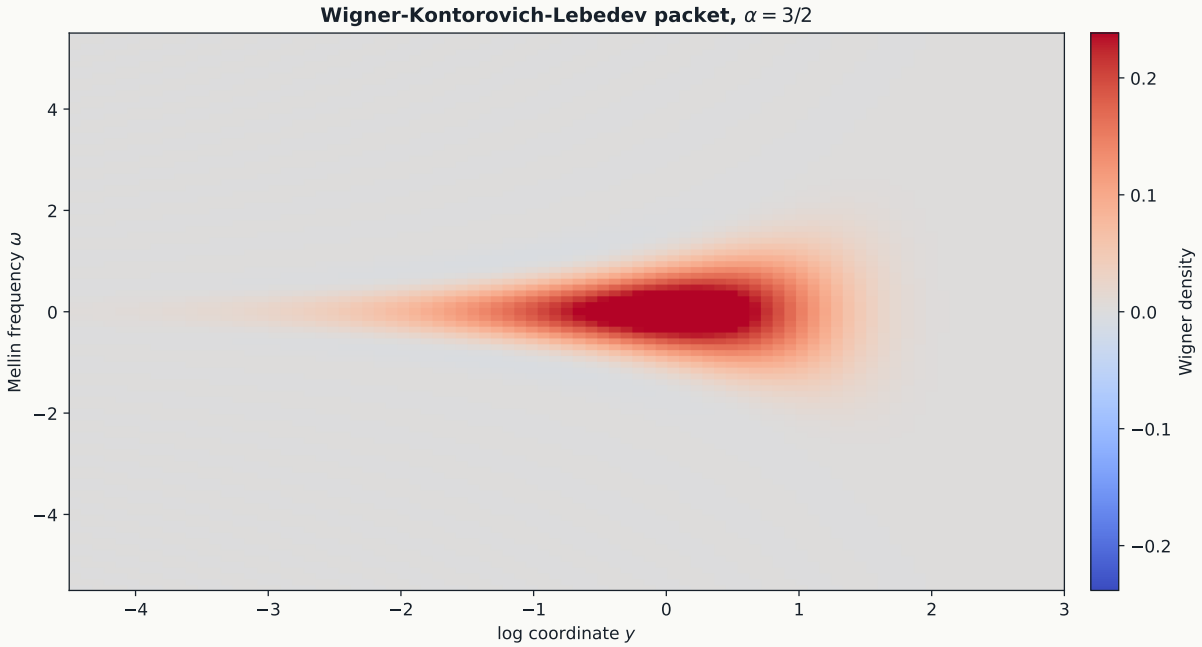


Figure 5.1: The Wigner–Kontorovich–Lebedev packet. The distribution is an exact Bessel function of imaginary order. Positive and negative regions encode the nonclassical phase-space interference of the packet.

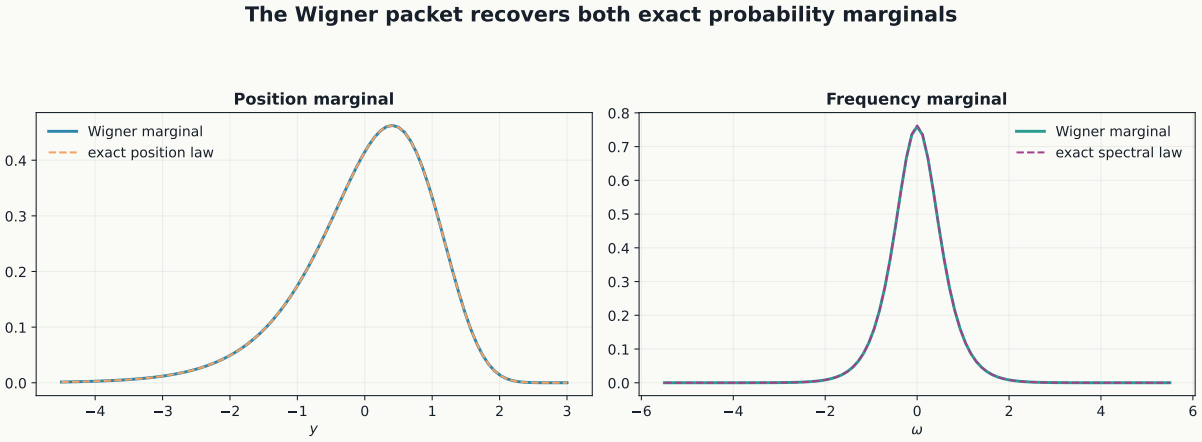


Figure 5.2: Wigner marginals. Numerical integration of the Bessel closed form reproduces both the exact log-Gamma position density and the exact Meixner spectral density.

Chapter 6

Uncertainty and quantum geometry

Let

$$Yf(y) = yf(y), \qquad Pf(y) = -\mathrm{i}f'(y).$$

Theorem 6.1 (Exact moments and uncertainty)**.** *For the state g_α,*

$$\boxed{\mathbb{E}Y = \psi(\alpha), \qquad \operatorname{Var} Y = \psi_1(\alpha),}$$

while

$$\boxed{\mathbb{E}P = 0, \qquad \operatorname{Var} P = \frac{\alpha}{4}.}$$

Consequently,

$$\boxed{\Delta Y\,\Delta P = \frac{1}{2}\sqrt{\alpha\psi_1(\alpha)} > \frac{1}{2}.}$$

Moreover,

$$\Delta Y\,\Delta P \longrightarrow \frac{1}{2}$$

as $\alpha \to \infty$.

Proof. The variable $U = \mathrm{e}^Y$ has the Gamma$(\alpha, 1)$ law. Its logarithmic moments give

$$\mathbb{E}Y = \psi(\alpha), \qquad \operatorname{Var} Y = \psi_1(\alpha).$$

For momentum,

$$\frac{g'_\alpha(y)}{g_\alpha(y)} = \frac{\alpha - \mathrm{e}^y}{2}.$$

The packet is real, so $\mathbb{E}P = 0$. Integration by parts gives

$$\mathbb{E}P^2 = \int |g'_\alpha(y)|^2 \,\mathrm{d}y = \frac{1}{4}\mathbb{E}(\alpha - U)^2 = \frac{\alpha}{4}.$$

The strict inequality follows from $\psi_1(\alpha) > 1/\alpha$. The asymptotic expansion $\psi_1(\alpha) = \alpha^{-1} + O(\alpha^{-2})$ gives the limit. □

α	$\operatorname{Var} Y$	$\operatorname{Var} P$	$\Delta Y\,\Delta P$	$\mathcal{I}_{\eta\eta}$
0.50	4.934802	0.125000	0.785398	19.739209
0.75	2.541880	0.187500	0.690364	10.167519
1.00	1.644934	0.250000	0.641275	6.579736
1.50	0.934802	0.375000	0.592073	3.739209
2.00	0.644934	0.500000	0.567862	2.579736
3.00	0.394934	0.750000	0.544243	1.579736
5.00	0.221323	1.250000	0.525979	0.885292
10.00	0.105166	2.500000	0.512753	0.420665

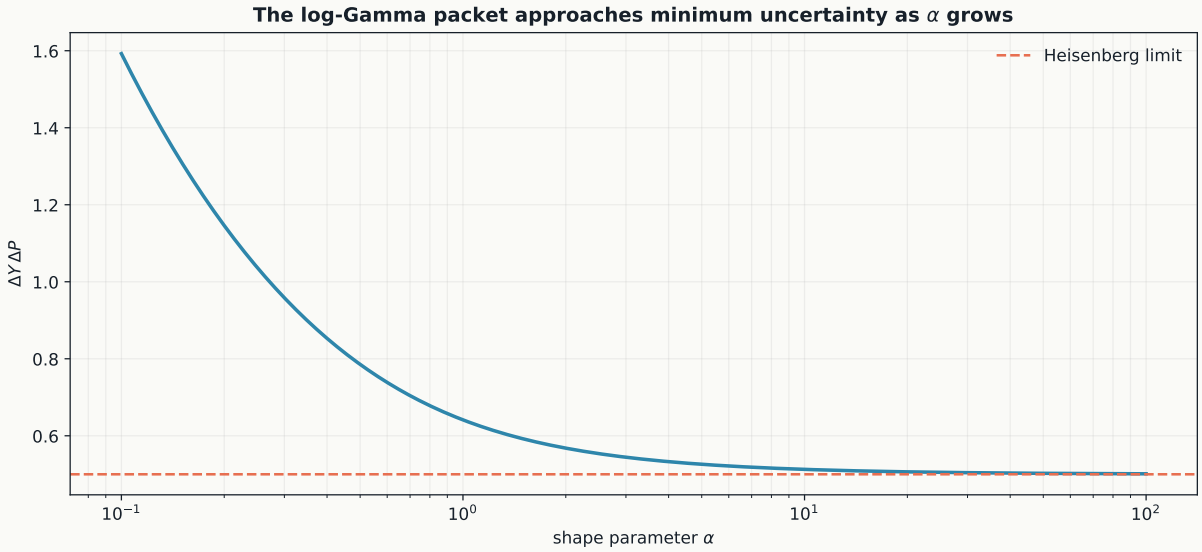


Figure 6.1: Exact uncertainty product. The packet is not a Gaussian at finite shape, but after centering and large-α scaling it approaches the minimum-uncertainty regime.

Define the centered Weyl orbit

$$g_{u,\eta}(y) = \mathrm{e}^{\mathrm{i}\eta(y-u/2)} g_\alpha(y-u).$$

Proposition 6.2 (Constant quantum geometric tensor)**.** *In coordinates (u,η), the Fubini–Study metric is constant and diagonal:*

$$\boxed{g_{uu} = \frac{\alpha}{4}, \qquad g_{\eta\eta} = \psi_1(\alpha), \qquad g_{u\eta} = 0.}$$

The Berry curvature is

$$\boxed{\mathcal{F}_{u\eta} = 1}$$

for the chosen Weyl convention. The quantum Fisher information matrix is

$$\boxed{\mathcal{I} = \mathrm{diag}(\alpha, 4\psi_1(\alpha))\,.}$$

Proof. Translation is generated by P and modulation by Y. The real part of the quantum geometric tensor is the covariance matrix of these generators. The mixed covariance vanishes because g_α is real and the centered symmetric derivative is odd under integration by parts. The imaginary part is one half of the canonical commutator $[Y,P] = \mathrm{i}I$. Multiplying the metric by four gives the quantum Fisher information. □

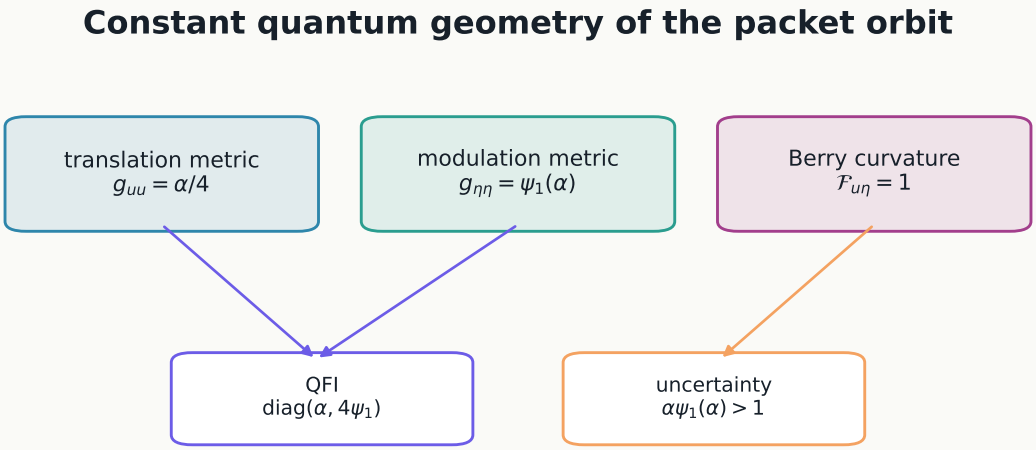


Figure 6.2: Quantum geometry of the Weyl orbit. Translation and modulation inherit exact metric coefficients from the packet variances. The canonical commutator supplies the constant symplectic curvature.

Part III

Orthogonal packet bases and one ladder algebra

Chapter 7

Laguerre packets on logarithmic position space

For $n \geq 0$, define

$$\boxed{\Lambda_{n,\alpha}(y) = \sqrt{\frac{n!}{\Gamma(n+\alpha)}}\, \mathrm{e}^{\alpha y/2 - \mathrm{e}^y/2} L_n^{(\alpha-1)}(\mathrm{e}^y).}$$

The ground packet is

$$\Lambda_{0,\alpha} = g_\alpha.$$

Theorem 7.1 (Orthonormality)**.**

$$\boxed{\int_{\mathbb{R}} \Lambda_{n,\alpha}(y)\Lambda_{m,\alpha}(y)\,\mathrm{d}y = \delta_{nm}.}$$

Proof. Set $x = \mathrm{e}^y$. Then $\mathrm{d}y = \mathrm{d}x/x$, and the inner product becomes the standard Laguerre integral

$$\frac{\sqrt{n!m!}}{\sqrt{\Gamma(n+\alpha)\Gamma(m+\alpha)}} \int_0^\infty x^{\alpha-1}\mathrm{e}^{-x} L_n^{(\alpha-1)}(x) L_m^{(\alpha-1)}(x)\,\mathrm{d}x.$$

The Laguerre orthogonality formula gives δ_{nm}. □

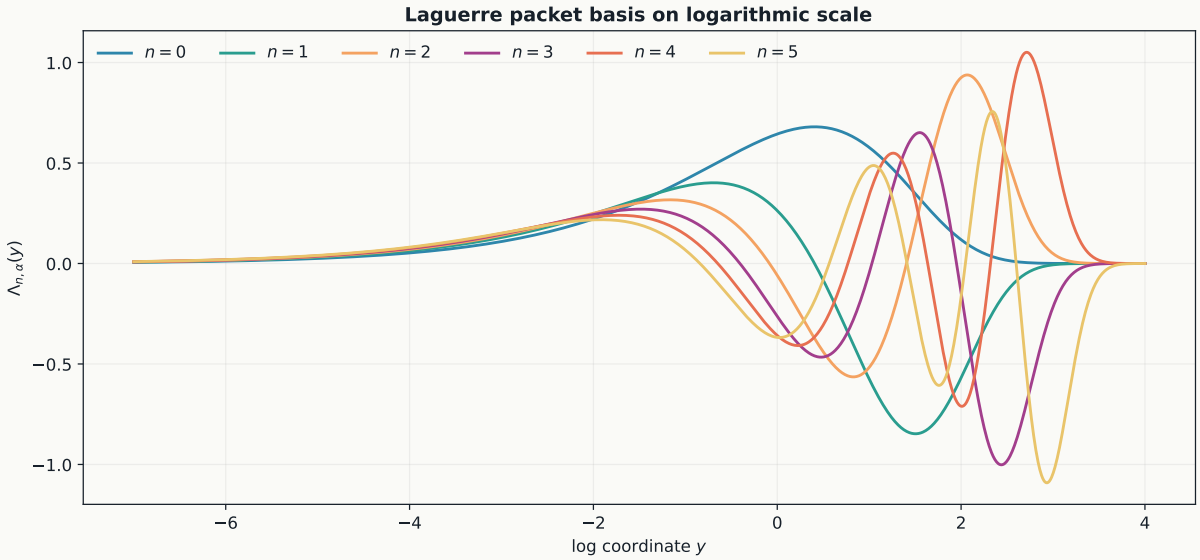


Figure 7.1: Laguerre packets in log coordinates. The packets form an exact orthonormal multioscillation family around the universal log-Gamma envelope.

Chapter 8

Meixner–Pollaczek packets in Mellin frequency

Let $p_{n,\alpha}$ denote the orthonormal polynomial sequence for the density

$$\rho_\alpha(\omega) = \frac{2^{\alpha-1}}{\pi\Gamma(\alpha)}\left|\Gamma\left(\frac{\alpha}{2}+\mathrm{i}\omega\right)\right|^2.$$

It is characterized by

$$p_{0,\alpha}=1, \qquad p_{1,\alpha}(\omega)=\frac{2\omega}{\sqrt{\alpha}},$$

and

$$\boxed{\omega p_{n,\alpha}(\omega)=\frac{1}{2}\sqrt{(n+1)(n+\alpha)}\,p_{n+1,\alpha}(\omega)+\frac{1}{2}\sqrt{n(n+\alpha-1)}\,p_{n-1,\alpha}(\omega).}$$

These are normalized Meixner–Pollaczek polynomials at angle $\pi/2$.

Theorem 8.1 (Laguerre–Meixner Fourier duality)**.** *For every $n \geq 0$,*

$$\boxed{\widehat{\Lambda}_{n,\alpha}(\omega)=\mathrm{i}^n\widehat{g}_\alpha(\omega)p_{n,\alpha}(\omega).}$$

Proof. Let

$$s=\frac{\alpha}{2}-\mathrm{i}\omega.$$

Using $x=\mathrm{e}^y$, the Fourier transform reduces to

$$\sqrt{\frac{n!}{2\pi\Gamma(n+\alpha)}}\int_0^\infty x^{s-1}\mathrm{e}^{-x/2}L_n^{(\alpha-1)}(x)\,\mathrm{d}x.$$

Introduce a generating variable z. The Laguerre generating function gives

$$\sum_{n\ge 0}\left[\int_0^\infty x^{s-1}\mathrm{e}^{-x/2}L_n^{(\alpha-1)}(x)\,\mathrm{d}x\right]z^n = 2^s\Gamma(s)(1-z)^{-\alpha/2-\mathrm{i}\omega}(1+z)^{-\alpha/2+\mathrm{i}\omega}.$$

After extracting the common ground-state factor, the remaining coefficients are the Meixner–Pollaczek generating coefficients with the phase i^n. The normalization is fixed by the Laguerre norms. This proves the formula. □

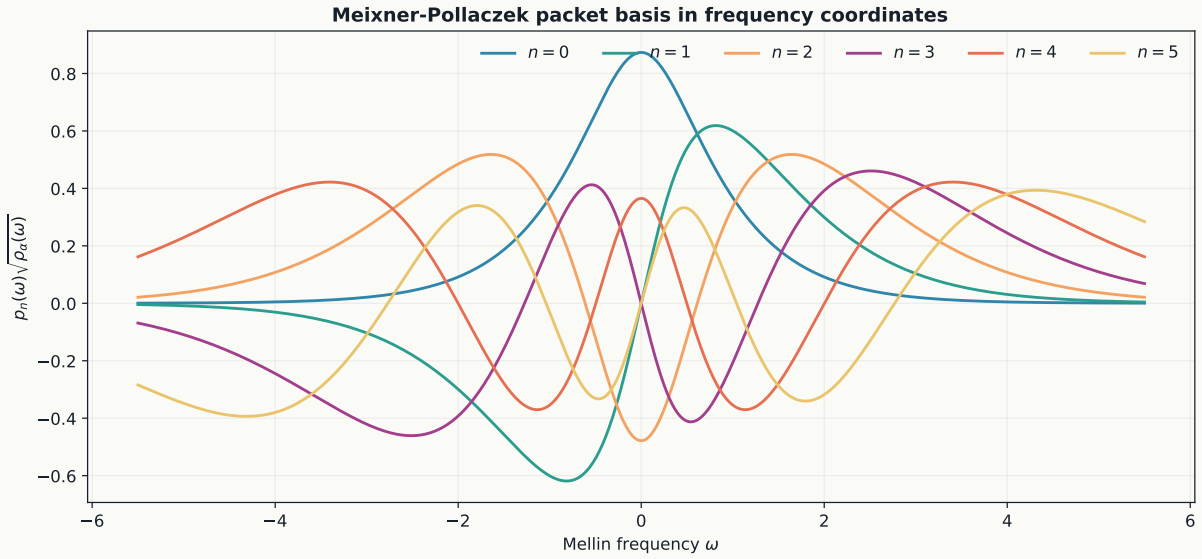


Figure 8.1: Meixner–Pollaczek packets. Multiplying each orthonormal polynomial by the square root of its Gamma density gives the frequency-side packet family.

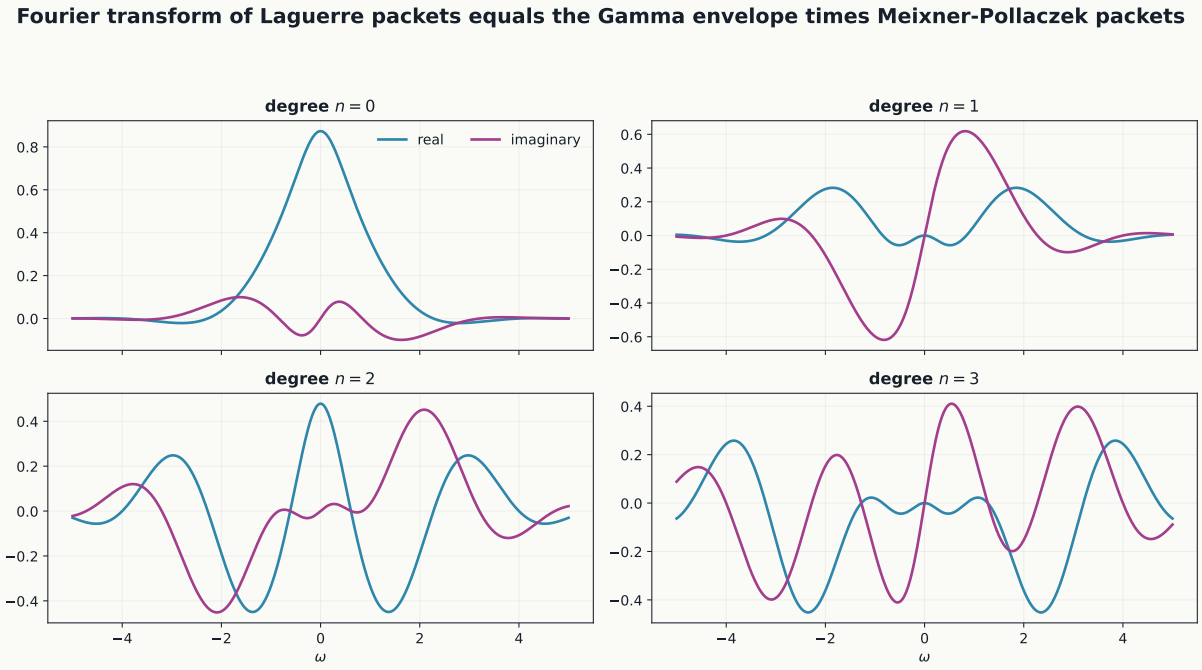


Figure 8.2: Exact Fourier duality. The real and imaginary parts alternate with degree through the phase i^n. The Gamma envelope is common to every packet.

Chapter 9

The common $\mathfrak{su}(1,1)$ ladder

Let $\{e_n\}_{n\geq 0}$ be an abstract orthonormal basis. Define

$$K_0 e_n = \left(n + \frac{\alpha}{2}\right) e_n,$$

$$K_+ e_n = \sqrt{(n+1)(n+\alpha)}\, e_{n+1},$$

and

$$K_- e_n = \sqrt{n(n+\alpha-1)}\, e_{n-1}.$$

Theorem 9.1 ($\mathfrak{su}(1,1)$ relations)**.** *On the finite linear span of the basis,*

$$\boxed{[K_0, K_+] = K_+, \qquad [K_0, K_-] = -K_-, \qquad [K_-, K_+] = 2K_0.}$$

The Casimir

$$\mathcal{Q} = K_0^2 - \frac{1}{2}(K_+ K_- + K_- K_+)$$

is

$$\boxed{\mathcal{Q} = \frac{\alpha}{2}\left(\frac{\alpha}{2} - 1\right) I.}$$

Proof. Apply each commutator to e_n. The coefficients reduce to elementary differences of the quadratic ladder weights. The same computation gives the constant Casimir value. □

In the Laguerre basis, multiplication by $x = \mathrm{e}^y$ is represented by

$$\boxed{J_L = 2K_0 - K_+ - K_-.}$$

In the Meixner–Pollaczek basis, multiplication by ω is represented by

$$\boxed{J_M = \frac{1}{2}(K_+ + K_-).}$$

Thus one representation carries two exact coordinate operators.

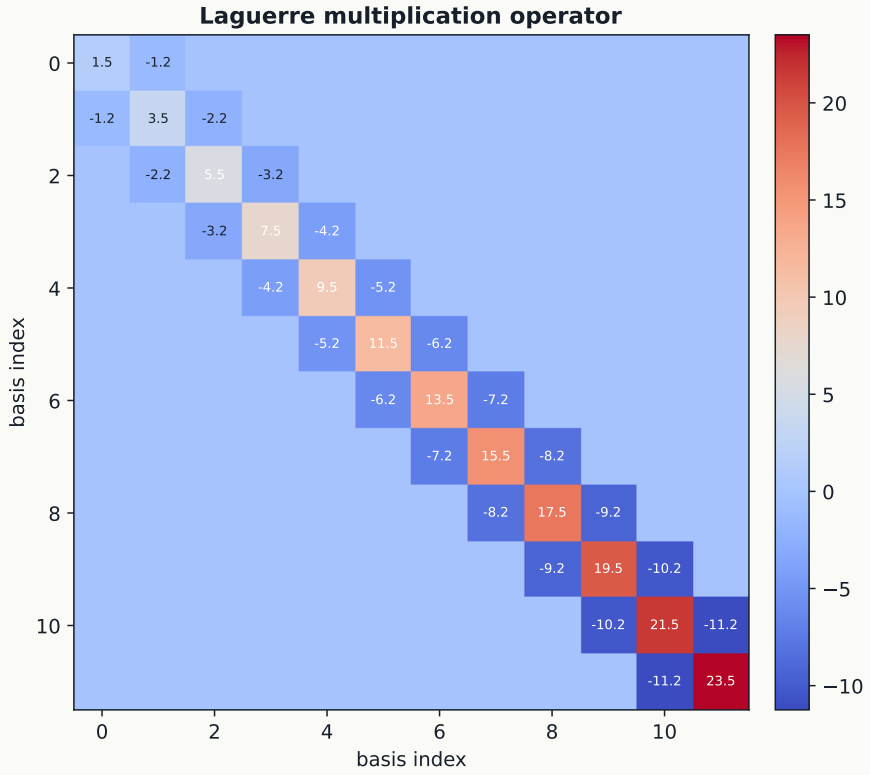


Figure 9.1: Laguerre Jacobi matrix. The position-side multiplication operator is tridiagonal in the logarithmic Laguerre packet basis.

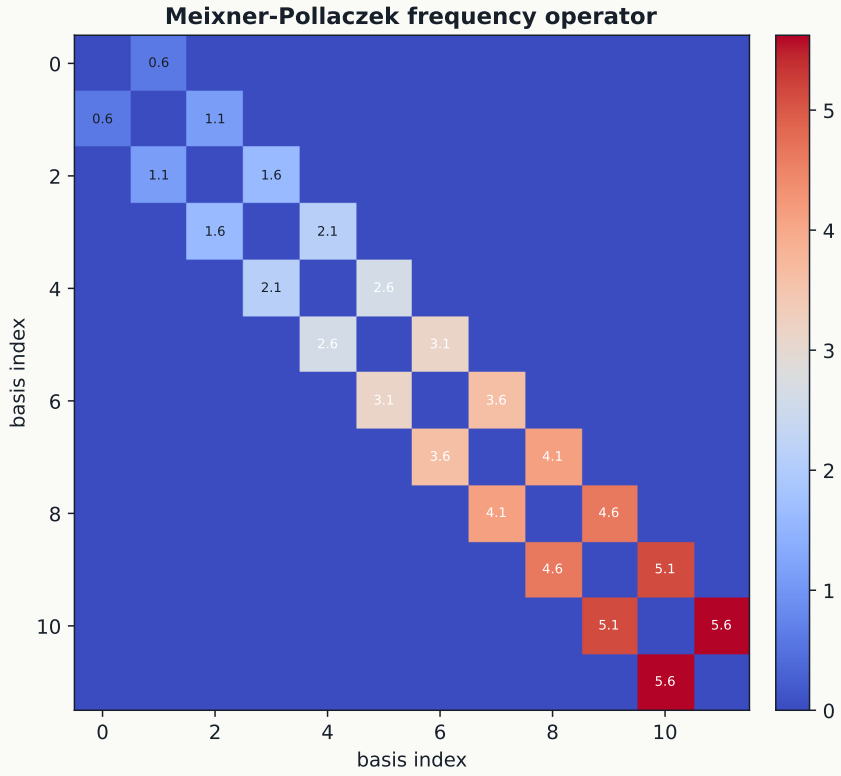


Figure 9.2: Meixner Jacobi matrix. The Mellin-frequency operator is another tridiagonal combination of the same ladder generators.

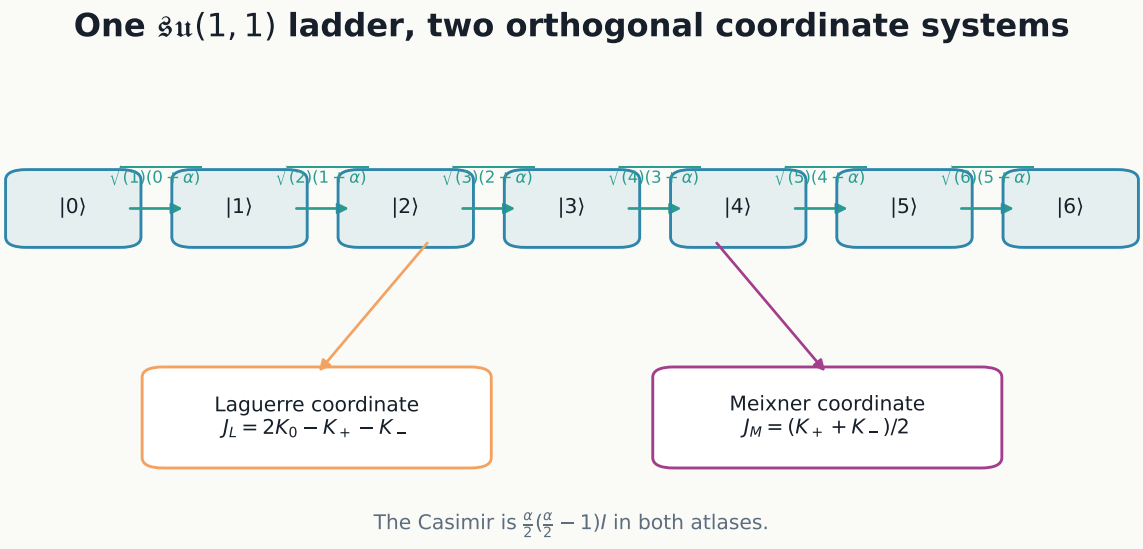


Figure 9.3: One ladder and two atlases. The Laguerre and Meixner operators are not unrelated recurrences. They are two projections of the same positive discrete-series ladder.

Part IV

Fractional depth, quantum realization, and the RH boundary

Chapter 10

Continuous packet depth

The integer-depth transfer from the earlier papers is

$$C_{a,\beta,\chi}(k,t) = \sum_{n\geq 1} \frac{\mu(n)\chi(n)}{n^{a+it}} \left(1 - \frac{1}{n^{\beta}}\right)^{k}.$$

For every real $r > 0$, define the continuous-depth extension

$$\boxed{C_{a,\beta,\chi}(r,t) = \sum_{n\geq 1} \frac{\mu(n)\chi(n)}{n^{a+it}} \left(1 - \frac{1}{n^{\beta}}\right)^{r}.}$$

Assume $a > 1$ so that all exchanges below are absolutely convergent.

Theorem 10.1 (Fractional binomial orbit)**.** *For $r > 0$,*

$$\boxed{C_{a,\beta,\chi}(r,t) = \sum_{j=0}^{\infty} (-1)^j \binom{r}{j} \frac{1}{L(a+\beta j+it,\chi)}.}$$

At integer $r = k$, the series terminates and recovers the finite-difference orbit.

Proof. For every $n \geq 2$,

$$\left(1 - n^{-\beta}\right)^r = \sum_{j \geq 0} (-1)^j \binom{r}{j} n^{-\beta j}.$$

The generalized binomial coefficients are absolutely summable for $r > 0$. The $n = 1$ packet vanishes on the left and the binomial sum at one is zero. Absolute convergence for $a > 1$ permits interchange of n and j. The inner Dirichlet series is the reciprocal L-function. □

Theorem 10.2 (Fractional-depth generator)**.**

$$\boxed{\frac{\partial}{\partial r} C_{a,\beta,\chi}(r,t) = -\sum_{m=1}^{\infty} \frac{1}{m} C_{a+m\beta,\beta,\chi}(r,t).}$$

Proof. Differentiate the factor $(1 - n^{-\beta})^r$ and use

$$\log(1 - n^{-\beta}) = -\sum_{m \geq 1} \frac{n^{-m\beta}}{m}.$$

Absolute convergence justifies the exchange of the sums and derivative. □

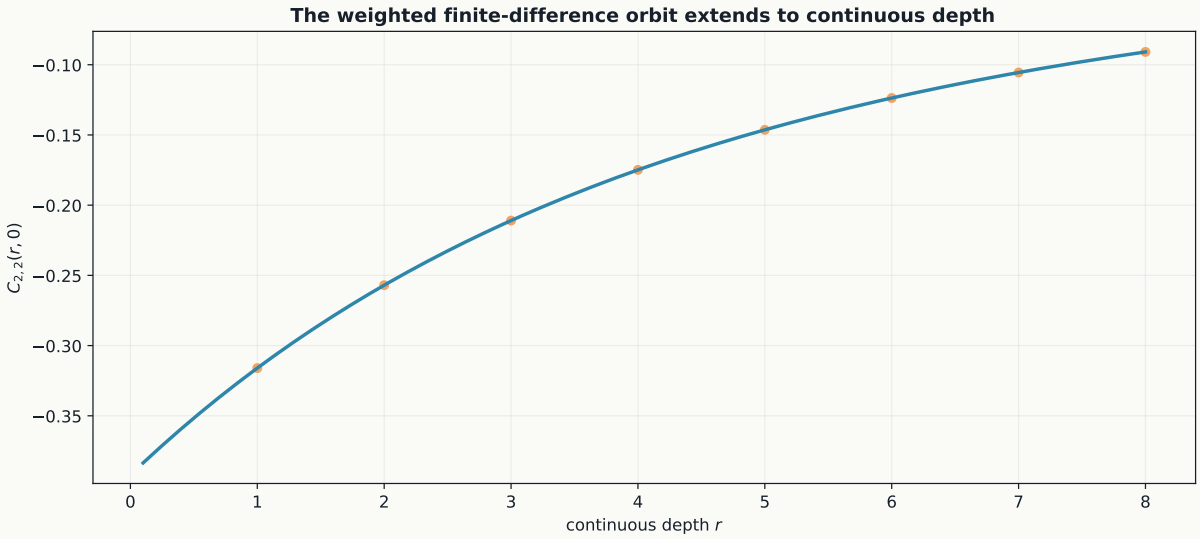


Figure 10.1: Continuous depth. The integer Bernoulli finite-difference orbit is embedded into a smooth depth evolution generated by an infinite parameter-shift operator.

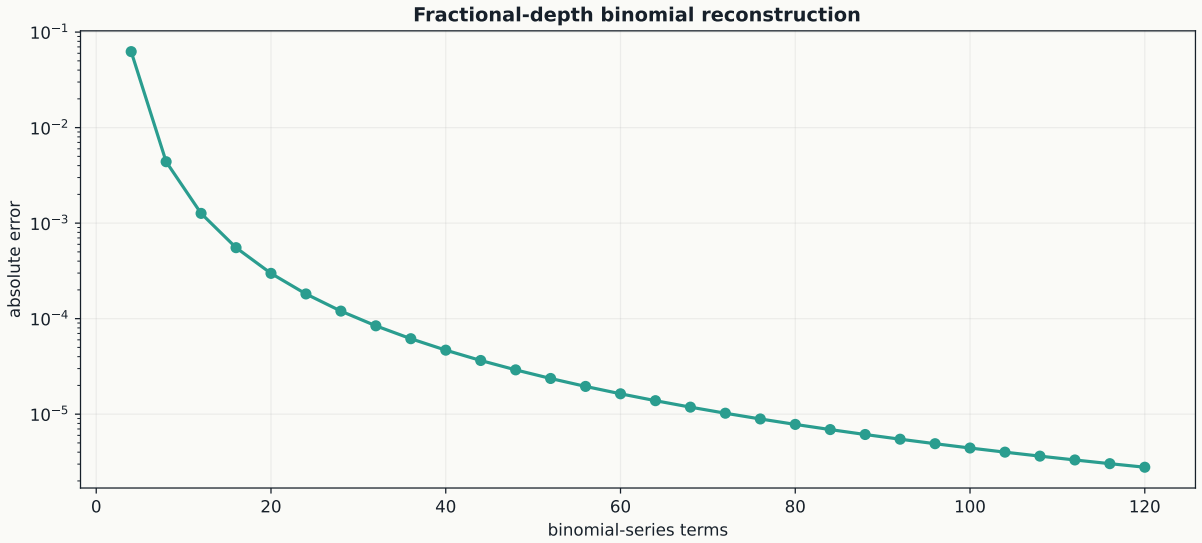


Figure 10.2: Fractional binomial reconstruction. The infinite reciprocal-zeta expansion converges to the direct Möbius packet calculation for the noninteger depth $r = 5/2$.

Chapter 11

Finite arithmetic convergence

The theorem at the beginning of the paper is asymptotic. The accompanying program evaluates the finite squarefree kernel directly and compares it with $\mathfrak{A}_\alpha$.

base depth	kernel error	real part	imaginary part
400	1.128878e-02	0.41019069	-0.01236909
1200	1.031104e-02	0.39292436	-0.02278309
4000	3.892217e-03	0.39764030	-0.01093128
12000	2.704832e-03	0.40156866	-0.01566558

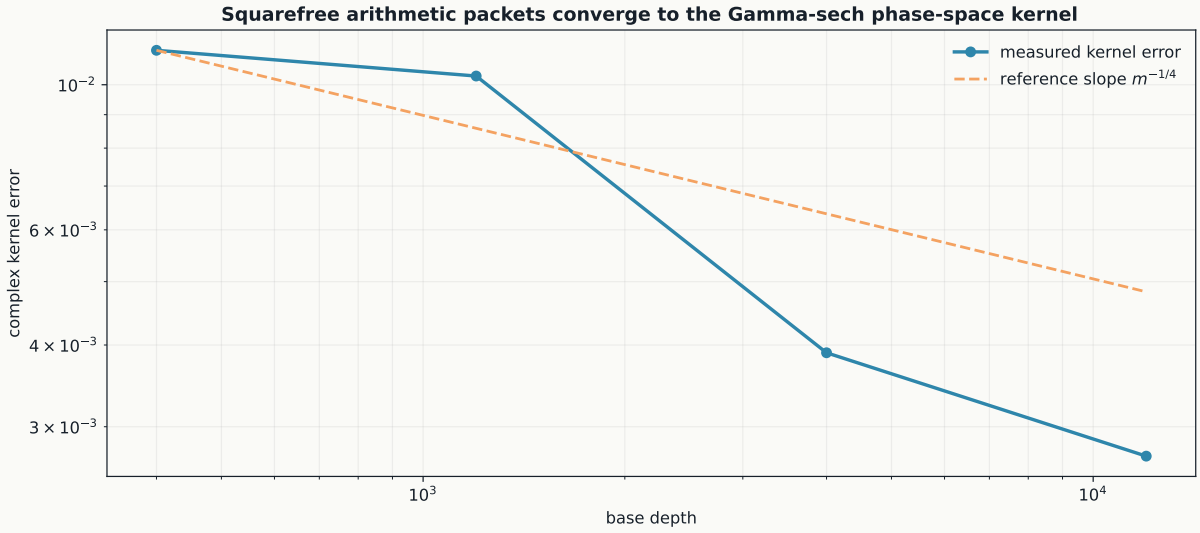


Figure 11.1: Arithmetic convergence. The finite squarefree packet moves toward the universal Gamma–sech kernel as the active scale grows. The displayed reference line is only a visual slope guide.

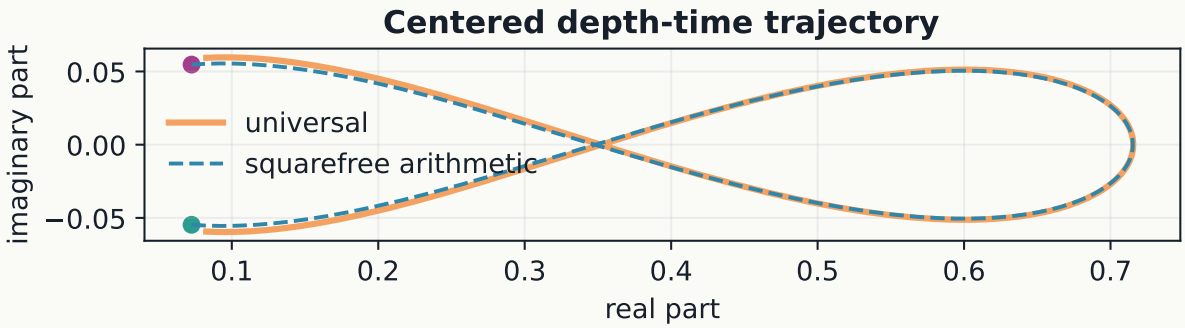


Figure 11.2: Complex trajectory comparison. After centering, the finite arithmetic correlation and the universal phase-space curve nearly coincide over the tested time interval.

Boundary statement 11.1 (What the arithmetic limit does not prove). The universal limit controls the positive squarefree kernel. It does not control the signed principal Möbius amplitude at time zero. The latter can be much smaller than the positive norm because of arithmetic cancellation. Establishing the required subpolynomial coherence remains the RH-equivalent problem.

Chapter 12

Quantum and signal realization

On the number basis, define

$$H\,|n\rangle = \log n\,|n\rangle\,, \qquad K_\beta\,|n\rangle = -\log(1-n^{-\beta})\,|n\rangle\,,$$

and

$$\mathcal{M}\,|n\rangle = \mu(n)\,|n\rangle\,.$$

The first operator is the logarithmic Hamiltonian. The second is the depth generator. The square $\mathcal{M}^2$ is the projector onto the squarefree number packets. All three operators are diagonal and commute.

For the amplitude vector

$$|\phi_a\rangle = \sum_{n\geq 2} n^{-a}\,|n\rangle\,,$$

the two-depth overlap

$$\boxed{\langle\phi_a|\,\mathrm{e}^{-\ell K_\beta}\mathcal{M}^2\mathrm{e}^{-\mathrm{i}tH}\mathrm{e}^{-kK_\beta}\,|\phi_a\rangle = \mathcal{K}_{a,\beta}(k,\ell,t)}$$

produces the depth-time kernel, with the excluded $n=1$ packet treated separately at zero depth. In the large-depth limit, this number-basis overlap becomes the Gabor ambiguity function of g_α.

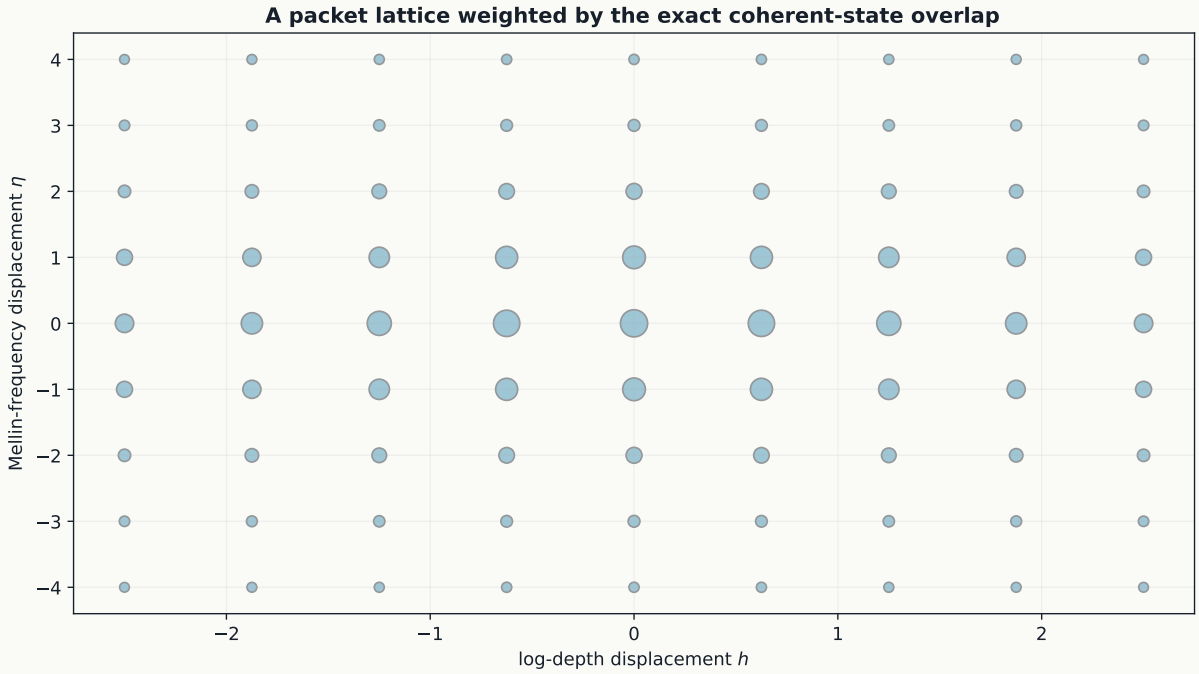


Figure 12.1: Packet lattice. Circle area is proportional to the exact overlap magnitude. The generated function supplies a natural nonuniform coherent-state geometry in log depth and Mellin frequency.

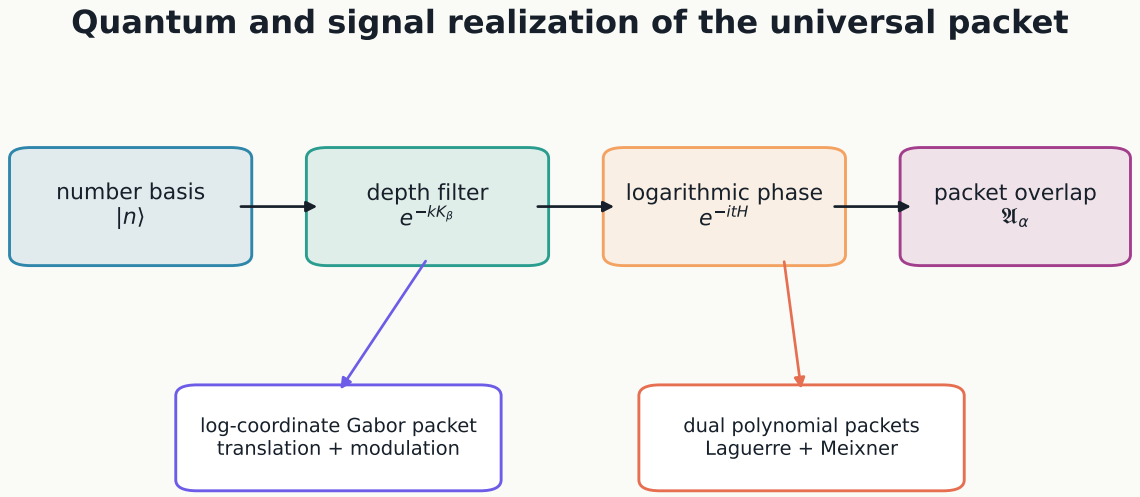


Figure 12.2: Quantum and signal path. The logarithmic Hamiltonian supplies the time phase. The depth filter supplies scale selection. Their centered overlap converges to the universal log-Gamma packet geometry.

The external quantum construction motivating this coordinate uses the same logarithmic spectrum and implements phases $n^{-\mathrm{i}t}$ through a logarithm oracle. The present packet adds the commuting scale generator K_β and studies a different positive overlap.

Chapter 13

The remaining RH boundary

For the principal parameters, define

$$c_k = \sum_{n\geq 1} \frac{\mu(n)}{n^2}\left(1-\frac{1}{n^2}\right)^k$$

and

$$E_k = \sum_{n\geq 1} \frac{\mu(n)^2}{n^4}\left(1-\frac{1}{n^2}\right)^{2k}.$$

The preceding papers prove

$$E_k \sim \frac{3}{2^{5/2}\pi^{3/2}} k^{-3/2}.$$

Define the packet coherence

$$\Gamma_k = \frac{|c_k|^2}{E_k}.$$

Using the classical sequential criterion,

$$\boxed{\mathrm{RH} \iff \Gamma_k = O_\varepsilon(k^\varepsilon) \quad \text{for every } \varepsilon > 0.}$$

The present paper solves the positive large-depth phase-space geometry exactly. It shows that the positive kernel has a zero-free Gabor limit, a complete orthogonal basis, a universal Wigner function, and a rigid ladder algebra. None of those positive facts forces the signed Möbius packet to have small coherence at the distinguished time $t = 0$.

Conjecture 13.1 (Principal Möbius coherence)**.** *For every $\varepsilon > 0$, there is a constant C_ε such that*

$$\boxed{|c_k|^2 \le C_\varepsilon k^\varepsilon E_k.}$$

This is stated as the remaining target, not as a theorem.

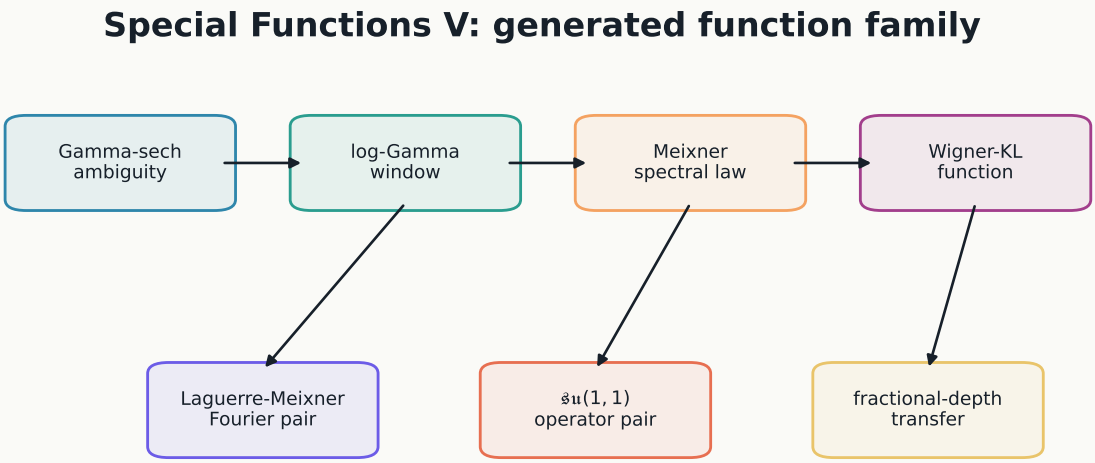


Figure 13.1: The generated family. The Gamma–sech ambiguity function, log-Gamma window, Meixner spectrum, Wigner–Bessel function, Laguerre–Meixner Fourier pair, ladder algebra, and fractional-depth transfer are exact coordinates of one universal packet system.

Part V

Verification, formula registry, and source

Chapter 14

Verification ledger

Certificate	Value
Built-in checks	19 of 19 passed
Ambiguity formula error	$6.704e-14$
Laguerre–Meixner Fourier error	$3.273e-12$
Meixner Gram error	$4.441e-15$
Laguerre Gram error	$3.132e-10$
$\mathfrak{su}(1,1)$ commutator error	$5.684e-14$
Poschl–Teller residual	$5.091e-08$
Fractional-depth error	$2.661e-10$

Table 14.1: Built-in verification ledger.

Check	Result	Detail
window normalization alpha=0.75	pass	integral=0.9999999999999
window normalization alpha=1.5	pass	integral=1
window normalization alpha=3.0	pass	integral=1
window normalization alpha=8.0	pass	integral=1
Gamma-sech ambiguity	pass	max_error=6.704e-14
Laguerre-Meixner Fourier duality	pass	max_error=3.273e-12
Meixner spectral density normalization	pass	integral=1
strict uncertainty alpha=0.75	pass	product=0.69036398656
strict uncertainty alpha=1.5	pass	product=0.592073327557
strict uncertainty alpha=3.0	pass	product=0.544243098382
strict uncertainty alpha=10.0	pass	product=0.512753195216
Meixner-Pollaczek orthogonality	pass	max_error=4.441e-15
Laguerre packet orthogonality	pass	max_error=3.132e-10
su(1,1) commutators	pass	max_error=5.684e-14
su(1,1) Casimir	pass	max_error=2.842e-14
Poschl-Teller equation	pass	max_error=5.091e-08
arithmetic joint-kernel convergence	pass	error_at_12000=2.705e-03
fractional-depth binomial orbit	pass	error=2.661e-10
real phase-space nonvanishing sample	pass	minimum=2.475e-06

The independent test suite rederives normalization, moments, ambiguity, the shape shift, spectral normalization, the characteristic function, Laguerre–Meixner duality, both orthogonalities, the ladder algebra, uncertainty, fractional depth, arithmetic Hermitian symmetry, and arithmetic convergence. It also verifies the figure inventory and the machine-readable result ledger.

Chapter 15

Formula registry

Object	Exact formula
Log-Gamma packet	$g_\alpha(y) = \Gamma(\alpha)^{-1/2} \exp(\alpha y/2 - \mathrm{e}^y/2)$
Ambiguity function	$\mathfrak{A}_\alpha(h,\eta) = \Gamma(\alpha+\mathrm{i}\eta)\Gamma(\alpha)^{-1} \operatorname{sech}^{\alpha+\mathrm{i}\eta}(h/2)$
Fourier transform	$\widehat{g}_\alpha(\omega) = 2^{\alpha/2-\mathrm{i}\omega}\Gamma(\alpha/2-\mathrm{i}\omega)/\sqrt{2\pi\Gamma(\alpha)}$
Spectral density	$\rho_\alpha(\omega) = 2^{\alpha-1}\lvert\Gamma(\alpha/2+\mathrm{i}\omega)\rvert^2/(\pi\Gamma(\alpha))$
Wigner function	$W_\alpha(y,\omega) = 2\mathrm{e}^{\alpha y}K_{2\mathrm{i}\omega}(\mathrm{e}^y)/(\pi\Gamma(\alpha))$
Uncertainty	$\Delta Y \Delta P = \frac{1}{2}\sqrt{\alpha\psi_1(\alpha)}$
Laguerre packet	$\Lambda_{n,\alpha}(y) = \sqrt{n!/\Gamma(n+\alpha)}\mathrm{e}^{\alpha y/2-\mathrm{e}^y/2}L_n^{(\alpha-1)}(\mathrm{e}^y)$
Fourier duality	$\widehat{\Lambda}_{n,\alpha} = \mathrm{i}^n \widehat{g}_\alpha p_{n,\alpha}$
Laguerre operator	$J_L = 2K_0 - K_+ - K_-$
Meixner operator	$J_M = (K_+ + K_-)/2$
Fractional depth	$C(r,t) = \sum_{j\ge0}(-1)^j\binom{r}{j}L(a+\beta j+\mathrm{i}t,\chi)^{-1}$

Chapter 16

Relation to established function theory

The Meixner–Pollaczek polynomials, Laguerre polynomials, Gabor ambiguity function, Wigner distribution, Kontorovich–Lebedev kernel, and positive discrete series of $\mathrm{SU}(1,1)$ are classical objects. This paper does not claim their invention.

The structural contribution is the exact generated bridge:

$$\boxed{\begin{array}{c}\text{squarefree depth-time packet} \longrightarrow \text{log-Gamma Gabor window} \longrightarrow \text{Gamma–sech ambiguity} \\ \longrightarrow \text{Laguerre–Meixner duality} \longrightarrow \mathfrak{su}(1,1)\end{array}}$$

The arithmetic limit fixes the parameter α, the centering, the carrier phase, and the normalization. The same parameter then controls the ambiguity function, uncertainty, polynomial recurrence, Casimir, and fractional-depth geometry.

Bibliography

[1] S. Wei, Y. Zhai, Q. Lu, et al., "The Riemann Hypothesis manifested in dynamical quantum phase transitions," *Nature Communications*, article in press, 2026.

[2] L. Báez-Duarte, "A sequential Riesz-like criterion for the Riemann hypothesis," *International Journal of Mathematics and Mathematical Sciences*, 2005.

[3] R. Koekoek, P. A. Lesky, and R. F. Swarttouw, *Hypergeometric Orthogonal Polynomials and Their q-Analogues*, Springer, 2010.

[4] G. B. Folland, *Harmonic Analysis in Phase Space*, Princeton University Press, 1989.

[5] N. Ja. Vilenkin and A. U. Klimyk, *Representation of Lie Groups and Special Functions*, volume 1, Kluwer, 1991.

[6] NIST Digital Library of Mathematical Functions, chapters on Gamma functions, orthogonal polynomials, and integral transforms.

[7] M. I. Kontorovich and N. N. Lebedev, classical papers on the transform generated by modified Bessel functions of imaginary order.

Appendix A

Complete Python implementation

The following single program generates every figure, every numerical table, the built-in certificate ledger, and the machine-readable results used in this paper.

```
from __future__ import annotations

import csv
import json
import math
from dataclasses import dataclass
from functools import lru_cache
from pathlib import Path
from typing import Iterable

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.patches import FancyArrowPatch, FancyBboxPatch, Rectangle
import mpmath as mp
import numpy as np
from scipy.integrate import quad
from scipy.special import digamma, eval_genlaguerre, gamma, gammaln, loggamma, polygamma

mp.mp.dps = 80

ROOT = Path(__file__).resolve().parent
FIGURES = ROOT / "figures"
RESULTS = ROOT / "results"
TABLES = ROOT / "tables"
GENERATED = ROOT / "generated_tables"
for directory in (FIGURES, RESULTS, TABLES, GENERATED):
    directory.mkdir(parents=True, exist_ok=True)

INK = "#17202A"
BLUE = "#2E86AB"
TEAL = "#2A9D8F"
ORANGE = "#F4A261"
MAGENTA = "#A23E8C"
RED = "#E76F51"
YELLOW = "#E9C46A"
PURPLE = "#6C5CE7"
MUTED = "#5D6D7E"
PAPER = "#FAFAF7"
GRID = "#D5D8DC"
WHITE = "#FFFFFF"
COLORS = [BLUE, TEAL, ORANGE, MAGENTA, RED, YELLOW, PURPLE]

plt.rcParams.update(
    {
        "figure.facecolor": PAPER,
        "axes.facecolor": PAPER,
```

```
        "axes.edgecolor": INK,
        "axes.labelcolor": INK,
        "xtick.color": INK,
        "ytick.color": INK,
        "text.color": INK,
        "font.size": 10.5,
        "axes.titlesize": 12.2,
        "axes.titleweight": "bold",
        "legend.frameon": False,
        "savefig.bbox": "tight",
        "savefig.facecolor": PAPER,
    }
)


def save_figure(fig: plt.Figure, stem: str) -> None:
    fig.savefig(FIGURES / f"{stem}.pdf")
    fig.savefig(FIGURES / f"{stem}.png", dpi=220)
    plt.close(fig)


def diagram_axes(figsize: tuple[float, float] = (12.6, 6.1)) -> tuple[plt.Figure, plt.Axes]:
    fig, ax = plt.subplots(figsize=figsize)
    ax.set_xlim(0, 1)
    ax.set_ylim(0, 1)
    ax.axis("off")
    return fig, ax


def box(
    ax: plt.Axes,
    xy: tuple[float, float],
    width: float,
    height: float,
    text: str,
    face: str = WHITE,
    edge: str = INK,
    fontsize: float = 10.8,
) -> None:
    patch = FancyBboxPatch(
        xy,
        width,
        height,
        boxstyle="round,pad=.012,rounding_size=.018",
        facecolor=face,
        edgecolor=edge,
        linewidth=1.7,
    )
    ax.add_patch(patch)
    ax.text(xy[0] + width / 2, xy[1] + height / 2, text, ha="center", va="center", fontsize=fontsize)


def arrow(
    ax: plt.Axes,
    start: tuple[float, float],
    end: tuple[float, float],
    color: str = INK,
    text: str | None = None,
    text_offset: tuple[float, float] = (0.0, 0.03),
) -> None:
    patch = FancyArrowPatch(start, end, arrowstyle="-|>", mutation_scale=13, linewidth=1.6, color=color)
    ax.add_patch(patch)
    if text:
        ax.text(
            (start[0] + end[0]) / 2 + text_offset[0],
            (start[1] + end[1]) / 2 + text_offset[1],
            text,
            ha="center",
            va="center",
            fontsize=9.8,
            color=color,
```

```
        )


# ---------------------------------------------------------------------------
# Log-Gamma packet window and generated special functions
# ---------------------------------------------------------------------------


def log_gamma_window(alpha: float, y: np.ndarray | float) -> np.ndarray:
    values = np.asarray(y, dtype=float)
    return np.exp(0.5 * alpha * values - 0.5 * np.exp(values) - 0.5 * gammaln(alpha))


def log_gamma_density(alpha: float, y: np.ndarray | float) -> np.ndarray:
    values = np.asarray(y, dtype=float)
    return np.exp(alpha * values - np.exp(values) - gammaln(alpha))


def log_gamma_fourier(alpha: float, omega: np.ndarray | float) -> np.ndarray:
    values = np.asarray(omega, dtype=float)
    z = alpha / 2.0 - 1j * values
    log_value = (alpha / 2.0 - 1j * values) * math.log(2.0) + loggamma(z)
    log_value -= 0.5 * (math.log(2.0 * math.pi) + gammaln(alpha))
    return np.exp(log_value)


def meixner_density(alpha: float, omega: np.ndarray | float) -> np.ndarray:
    values = np.asarray(omega, dtype=float)
    return np.abs(log_gamma_fourier(alpha, values)) ** 2


def gamma_sech_kernel(alpha: float, h: np.ndarray | float, eta: np.ndarray | float) -> np.ndarray:
    h_values, eta_values = np.broadcast_arrays(np.asarray(h, dtype=float), np.asarray(eta, dtype=float))
    z = alpha + 1j * eta_values
    log_sech = -np.log(np.cosh(h_values / 2.0))
    return np.exp(loggamma(z) - gammaln(alpha) + z * log_sech)


def centered_ambiguity_numeric(alpha: float, h: float, eta: float) -> complex:
    integrand = lambda y: (
        log_gamma_window(alpha, y + h / 2)
        * log_gamma_window(alpha, y - h / 2)
        * np.exp(1j * eta * y)
    )
    real = quad(lambda yy: float(np.real(integrand(yy))), -20, 8, epsabs=2e-12, limit=300)[0]
    imag = quad(lambda yy: float(np.imag(integrand(yy))), -20, 8, epsabs=2e-12, limit=300)[0]
    return complex(real, imag)


def wigner_log_gamma(alpha: float, y: np.ndarray, omega: np.ndarray) -> np.ndarray:
    yy, ww = np.meshgrid(np.asarray(y, dtype=float), np.asarray(omega, dtype=float), indexing="ij")
    out = np.empty_like(yy)
    prefactor = 2.0 / (math.pi * float(mp.gamma(alpha)))
    for i in range(yy.shape[0]):
        x = float(np.exp(yy[i, 0]))
        for j in range(yy.shape[1]):
            out[i, j] = prefactor * math.exp(alpha * yy[i, j]) * float(mp.re(mp.besselk(2j * ww[i, j], x)))
    return out


def uncertainty_data(alpha: float) -> dict[str, float]:
    var_y = float(polygamma(1, alpha))
    var_p = alpha / 4.0
    return {
        "mean_y": float(digamma(alpha)),
        "var_y": var_y,
        "var_p": var_p,
        "product": math.sqrt(var_y * var_p),
        "qfi_u": alpha,
        "qfi_eta": 4.0 * var_y,
        "berry_curvature": 1.0,
```

```
    }


def meixner_pollaczek_orthonormal(alpha: float, degree: int, omega: np.ndarray | float) -> np.ndarray:
    x = np.asarray(omega, dtype=float)
    if degree == 0:
        return np.ones_like(x)
    p_prev = np.ones_like(x)
    p_curr = 2.0 * x / math.sqrt(alpha)
    if degree == 1:
        return p_curr
    for n in range(1, degree):
        b_np1 = 0.5 * math.sqrt((n + 1) * (n + alpha))
        b_n = 0.5 * math.sqrt(n * (n + alpha - 1))
        p_next = (x * p_curr - b_n * p_prev) / b_np1
        p_prev, p_curr = p_curr, p_next
    return p_curr


def laguerre_packet(alpha: float, degree: int, y: np.ndarray | float) -> np.ndarray:
    values = np.asarray(y, dtype=float)
    normalization = math.exp(0.5 * (gammaln(degree + 1) - gammaln(degree + alpha)))
    return (
        normalization
        * np.exp(0.5 * alpha * values - 0.5 * np.exp(values))
        * eval_genlaguerre(degree, alpha - 1.0, np.exp(values))
    )


def laguerre_packet_fourier(alpha: float, degree: int, omega: np.ndarray | float) -> np.ndarray:
    values = np.asarray(omega, dtype=float)
    return (
        (1j ** degree)
        * log_gamma_fourier(alpha, values)
        * meixner_pollaczek_orthonormal(alpha, degree, values)
    )


def laguerre_packet_fourier_numeric(alpha: float, degree: int, omega: float) -> complex:
    integrand = lambda y: laguerre_packet(alpha, degree, y) * np.exp(-1j * omega * y) / math.sqrt(2 * math.pi)
    real = quad(lambda yy: float(np.real(integrand(yy))), -35, 10, epsabs=2e-12, limit=450)[0]
    imag = quad(lambda yy: float(np.imag(integrand(yy))), -35, 10, epsabs=2e-12, limit=450)[0]
    return complex(real, imag)


# ---------------------------------------------------------------------------
# Ladder operators and exact matrix systems
# ---------------------------------------------------------------------------


def su11_matrices(alpha: float, dimension: int) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
    k0 = np.diag(np.arange(dimension, dtype=float) + alpha / 2.0)
    kp = np.zeros((dimension, dimension), dtype=float)
    for n in range(dimension - 1):
        kp[n + 1, n] = math.sqrt((n + 1) * (n + alpha))
    km = kp.T
    return k0, kp, km


def laguerre_jacobi(alpha: float, dimension: int) -> np.ndarray:
    k0, kp, km = su11_matrices(alpha, dimension)
    return 2.0 * k0 - kp - km


def meixner_jacobi(alpha: float, dimension: int) -> np.ndarray:
    _, kp, km = su11_matrices(alpha, dimension)
    return 0.5 * (kp + km)


# ---------------------------------------------------------------------------
# Arithmetic kernel and fractional-depth extension
```

```
# ----------------------------------------------------------------------------


@lru_cache(maxsize=12)
def mobius_sieve(nmax: int) -> np.ndarray:
    mu = np.zeros(nmax + 1, dtype=np.int8)
    mu[1] = 1
    primes: list[int] = []
    composite = np.zeros(nmax + 1, dtype=bool)
    for i in range(2, nmax + 1):
        if not composite[i]:
            primes.append(i)
            mu[i] = -1
        for p in primes:
            value = i * p
            if value > nmax:
                break
            composite[value] = True
            if i % p == 0:
                mu[value] = 0
                break
            mu[value] = -mu[i]
    return mu


@dataclass
class ArithmeticKernel:
    a: float
    beta: float
    n: np.ndarray
    weight: np.ndarray
    log_y: np.ndarray

    @property
    def alpha(self) -> float:
        return (2 * self.a - 1) / self.beta

    def kernel(self, k: float, ell: float, tau: float = 0.0) -> complex:
        depth = k + ell
        factor = np.exp(depth * self.log_y)
        phase = np.exp(-1j * tau * np.log(self.n))
        return complex(np.sum(self.weight * factor * phase))

    def normalized_centered(self, k: float, ell: float, tau: float) -> complex:
        eta = tau / self.beta
        numerator = self.kernel(k, ell, tau)
        denominator = math.sqrt(self.kernel(k, k, 0.0).real * self.kernel(ell, ell, 0.0).real)
        carrier = np.exp(1j * eta * math.log(2.0 * math.sqrt(k * ell)))
        return complex(carrier * numerator / denominator)


def build_arithmetic_kernel(a: float = 2.0, beta: float = 2.0, nmax: int = 600_000) -> ArithmeticKernel:
    mu = mobius_sieve(nmax)
    integers = np.arange(1, nmax + 1, dtype=np.int64)
    mask = mu[1:] != 0
    n = integers[mask].astype(float)
    y = 1.0 - n ** (-beta)
    log_y = np.full_like(y, -np.inf)
    positive = y > 0
    log_y[positive] = np.log(y[positive])
    weight = n ** (-2 * a)
    return ArithmeticKernel(a=a, beta=beta, n=n, weight=weight, log_y=log_y)


def universal_arithmetic_limit(alpha: float, beta: float, k: float, ell: float, tau: float) -> complex:
    h = math.log(k / ell)
    eta = tau / beta
    return complex(gamma_sech_kernel(alpha, h, eta))


def fractional_mobius_transfer(r: float, a: float = 2.0, beta: float = 2.0, nmax: int = 500_000) -> float:
```

```
    mu = mobius_sieve(nmax)
    integers = np.arange(2, nmax + 1, dtype=float)
    muv = mu[2:].astype(float)
    return float(np.sum(muv * integers ** (-a) * (1.0 - integers ** (-beta)) ** r))


def fractional_binomial_transfer(r: float, a: float = 2.0, beta: float = 2.0, terms: int = 120) -> mp.mpf:
    mp.mp.dps = 60
    total = mp.mpf("0")
    for j in range(terms):
        total += (-1) ** j * mp.binomial(r, j) / mp.zeta(a + beta * j)
    return total


# ------------------------------------------------------------------------
# Checks
# ------------------------------------------------------------------------


def perform_checks() -> tuple[list[dict[str, object]], dict[str, object]]:
    checks: list[dict[str, object]] = []

    def record(name: str, passed: bool, detail: str) -> None:
        checks.append({"check": name, "passed": bool(passed), "detail": detail})

    # Normalization and moment identities
    for alpha in (0.75, 1.5, 3.0, 8.0):
        integral = quad(lambda y: float(log_gamma_density(alpha, y)), -40, 12, epsabs=2e-12, limit=400)[0]
        record(f"window normalization alpha={alpha}", abs(integral - 1.0) < 2e-10, f"integral={integral:.13g}")

    # Exact ambiguity formula
    ambiguity_errors = []
    for h, eta in ((0.0, 0.7), (0.8, 0.5), (1.4, -1.1), (-0.6, 1.7)):
        numeric = centered_ambiguity_numeric(1.5, h, eta)
        exact = complex(gamma_sech_kernel(1.5, h, eta))
        ambiguity_errors.append(abs(numeric - exact))
    record("Gamma-sech ambiguity", max(ambiguity_errors) < 3e-9, f"max_error={max(ambiguity_errors):.3e}")

    # Fourier transform formula
    fourier_errors = []
    for degree in range(5):
        for omega in (-1.2, -0.3, 0.8):
            numeric = laguerre_packet_fourier_numeric(1.5, degree, omega)
            exact = complex(laguerre_packet_fourier(1.5, degree, omega))
            fourier_errors.append(abs(numeric - exact))
    record("Laguerre-Meixner Fourier duality", max(fourier_errors) < 2e-8, f"max_error={max(fourier_errors):.3e}")

    # Spectral density normalization and moments
    spectral_integral = quad(lambda w: float(meixner_density(1.5, w)), -20, 20, epsabs=1e-10, limit=300)[0]
    record("Meixner spectral density normalization", abs(spectral_integral - 1.0) < 2e-9, f"integral={
     spectral_integral:.13g}")

    # Uncertainty formulas
    for alpha in (0.75, 1.5, 3.0, 10.0):
        data = uncertainty_data(alpha)
        record(
            f"strict uncertainty alpha={alpha}",
            data["product"] > 0.5 and data["product"] < 0.8,
            f"product={data['product']:.12g}",
        )

    # Meixner-Pollaczek orthogonality
    nodes = np.linspace(-18, 18, 40001)
    rho = meixner_density(1.5, nodes)
    polys = np.vstack([meixner_pollaczek_orthonormal(1.5, n, nodes) for n in range(5)])
    gram = np.array([[np.trapezoid(rho * polys[i] * polys[j], nodes) for j in range(5)] for i in range(5)])
    mp_error = float(np.max(np.abs(gram - np.eye(5))))
    record("Meixner-Pollaczek orthogonality", mp_error < 2e-7, f"max_error={mp_error:.3e}")

    # Laguerre packet orthogonality
    y_grid = np.linspace(-15, 7, 60001)
```

```
    packets = np.vstack([laguerre_packet(1.5, n, y_grid) for n in range(5)])
    lag_gram = np.array([[np.trapezoid(packets[i] * packets[j], y_grid) for j in range(5)] for i in range(5)])
    lag_error = float(np.max(np.abs(lag_gram - np.eye(5))))
    record("Laguerre packet orthogonality", lag_error < 2e-7, f"max_error={lag_error:.3e}")

    # SU(1,1) commutators on the non-boundary block
    alpha = 1.5
    dim = 18
    k0, kp, km = su11_matrices(alpha, dim)
    interior = slice(0, dim - 2)
    c1 = (k0 @ kp - kp @ k0 - kp)[interior, interior]
    c2 = (k0 @ km - km @ k0 + km)[interior, interior]
    c3 = (km @ kp - kp @ km - 2 * k0)[interior, interior]
    comm_error = max(float(np.max(np.abs(c1))), float(np.max(np.abs(c2))), float(np.max(np.abs(c3))))
    record("su(1,1) commutators", comm_error < 2e-12, f"max_error={comm_error:.3e}")

    casimir = k0 @ k0 - 0.5 * (kp @ km + km @ kp)
    target = alpha / 2 * (alpha / 2 - 1)
    cas_error = float(np.max(np.abs(casimir[: dim - 2, : dim - 2] - target * np.eye(dim - 2))))
    record("su(1,1) Casimir", cas_error < 2e-12, f"max_error={cas_error:.3e}")

    # Poschl-Teller ODE
    h = np.linspace(-4, 4, 20001)
    eta = 0.7
    nu = 1.5 + 1j * eta
    f = gamma_sech_kernel(1.5, h, eta)
    dh = h[1] - h[0]
    fpp = (f[:-2] - 2 * f[1:-1] + f[2:]) / dh**2
    residual = -4 * fpp - nu * (nu + 1) / np.cosh(h[1:-1] / 2) ** 2 * f[1:-1] + nu**2 * f[1:-1]
    pt_error = float(np.max(np.abs(residual[10:-10])))
    record("Poschl-Teller equation", pt_error < 2e-6, f"max_error={pt_error:.3e}")

    # Arithmetic convergence to the universal kernel
    arithmetic = build_arithmetic_kernel(nmax=600_000)
    convergence_rows = []
    max_large_error = 0.0
    for scale in (400, 1200, 4000, 12000):
        k = float(scale)
        ell = float(4 * scale)
        tau = 2.4
        discrete = arithmetic.normalized_centered(k, ell, tau)
        universal = universal_arithmetic_limit(arithmetic.alpha, arithmetic.beta, k, ell, tau)
        error = abs(discrete - universal)
        max_large_error = error if scale == 12000 else max_large_error
        convergence_rows.append({"scale": scale, "error": error, "discrete_real": discrete.real, "discrete_imag":
 discrete.imag})
    record("arithmetic joint-kernel convergence", max_large_error < 7e-3, f"error_at_12000={max_large_error:.3e}")

    # Fractional-depth binomial formula
    frac_direct = fractional_mobius_transfer(2.5, nmax=5_000_000)
    frac_series = float(fractional_binomial_transfer(2.5, terms=5000))
    frac_error = abs(frac_direct - frac_series)
    record("fractional-depth binomial orbit", frac_error < 8e-9, f"error={frac_error:.3e}")

    # Zero-free real phase-space sample
    h_grid = np.linspace(-6, 6, 121)
    eta_grid = np.linspace(-8, 8, 121)
    hh, ee = np.meshgrid(h_grid, eta_grid, indexing="ij")
    values = gamma_sech_kernel(1.5, hh, ee)
    minimum = float(np.min(np.abs(values)))
    record("real phase-space nonvanishing sample", minimum > 0.0, f"minimum={minimum:.3e}")

    passed = sum(1 for check in checks if check["passed"])
    summary: dict[str, object] = {
        "built_in_checks": {"passed": passed, "total": len(checks)},
        "ambiguity_max_error": max(ambiguity_errors),
        "fourier_duality_max_error": max(fourier_errors),
        "meixner_gram_error": mp_error,
        "laguerre_gram_error": lag_error,
        "su11_commutator_error": comm_error,
        "su11_casimir_error": cas_error,
```

```
        "poschl_teller_error": pt_error,
        "fractional_depth_error": frac_error,
        "phase_space_minimum_sample": minimum,
        "arithmetic_convergence": convergence_rows,
        "uncertainty_alpha_1_5": uncertainty_data(1.5),
    }
    return checks, summary


# ---------------------------------------------------------------------------
# Figure generation
# ---------------------------------------------------------------------------


def generate_figures(summary: dict[str, object]) -> None:
    # 01 pipeline
    fig, ax = diagram_axes()
    ax.text(0.5, 0.94, "From arithmetic depth packets to a universal phase-space function", ha="center", fontsize=19,
      weight="bold")
    items = [
        (0.03, "squarefree\ndepth-time kernel", BLUE),
        (0.22, "large-depth\nGamma scaling", TEAL),
        (0.41, "log-Gamma\npacket window", ORANGE),
        (0.60, "Gamma-sech\nambiguity kernel", MAGENTA),
        (0.79, "Laguerre and\nMeixner duality", PURPLE),
    ]
    for x0, text, color in items:
        box(ax, (x0, 0.56), 0.16, 0.18, text, face=color + "20", edge=color, fontsize=12)
    for left, right in zip(items[:-1], items[1:]):
        arrow(ax, (left[0] + 0.16, 0.65), (right[0], 0.65), color=INK)
    box(ax, (0.18, 0.18), 0.25, 0.15, "exact ambiguity\nPoschl-Teller equation", face=WHITE, edge=TEAL, fontsize
     =11.5)
    box(ax, (0.57, 0.18), 0.25, 0.15, "SU(1,1) ladder\nand quantum geometry", face=WHITE, edge=PURPLE, fontsize
     =11.5)
    arrow(ax, (0.49, 0.56), (0.305, 0.33), color=TEAL)
    arrow(ax, (0.68, 0.56), (0.695, 0.33), color=PURPLE)
    ax.text(0.5, 0.07, "The limiting packet is a concrete Gabor window rather than an abstract asymptotic profile.",
      ha="center", color=MUTED, fontsize=11.5)
    save_figure(fig, "01_theory_pipeline")

    # 02 window family
    y = np.linspace(-7, 4, 1800)
    fig, ax = plt.subplots(figsize=(12.6, 5.3))
    for alpha, color in zip((0.75, 1.5, 3.0, 8.0), COLORS):
        ax.plot(y, log_gamma_window(alpha, y), lw=2.1, color=color, label=rf"$\alpha={alpha}$")
    ax.set_xlabel("log coordinate $y$")
    ax.set_ylabel(r"$g_\alpha(y)$")
    ax.set_title("Normalized log-Gamma packet windows")
    ax.grid(alpha=0.18)
    ax.legend(ncol=4)
    save_figure(fig, "02_log_gamma_windows")

    # 03 position density
    fig, ax = plt.subplots(figsize=(12.6, 5.3))
    for alpha, color in zip((0.75, 1.5, 3.0, 8.0), COLORS):
        ax.plot(y, log_gamma_density(alpha, y), lw=2.1, color=color, label=rf"$\alpha={alpha}$")
    ax.set_xlabel(r"$y=\log u$")
    ax.set_ylabel("probability density")
    ax.set_title("The packet position law is log-Gamma")
    ax.grid(alpha=0.18)
    ax.legend(ncol=4)
    save_figure(fig, "03_log_gamma_density")

    # 04 spectral density
    omega = np.linspace(-8, 8, 2000)
    fig, ax = plt.subplots(figsize=(12.6, 5.3))
    for alpha, color in zip((0.75, 1.5, 3.0, 8.0), COLORS):
        ax.plot(omega, meixner_density(alpha, omega), lw=2.1, color=color, label=rf"$\alpha={alpha}$")
    ax.set_xlabel(r"Mellin frequency $\omega$")
    ax.set_ylabel(r"$|\widehat g_\alpha(\omega)|^2$")
    ax.set_title("The dual packet law is the symmetric Meixner density")
```

```
    ax.grid(alpha=0.18)
    ax.legend(ncol=4)
    save_figure(fig, "04_meixner_density")

    # 05 autocorrelation
    h = np.linspace(-7, 7, 1600)
    fig, ax = plt.subplots(figsize=(12.6, 5.2))
    for alpha, color in zip((0.75, 1.5, 3.0, 8.0), COLORS):
        ax.plot(h, 1 / np.cosh(h / 2) ** alpha, lw=2.1, color=color, label=rf"$\alpha={alpha}$")
    ax.set_xlabel("log-depth difference $h$")
    ax.set_ylabel("normalized overlap")
    ax.set_title(r"Exact depth overlap $\operatorname{sech}^{\alpha}(h/2)$")
    ax.grid(alpha=0.18)
    ax.legend(ncol=4)
    save_figure(fig, "05_depth_autocorrelation")

    # 06 ambiguity magnitude
    h_grid = np.linspace(-5, 5, 181)
    eta_grid = np.linspace(-7, 7, 181)
    hh, ee = np.meshgrid(h_grid, eta_grid, indexing="xy")
    amb = gamma_sech_kernel(1.5, hh, ee)
    fig, ax = plt.subplots(figsize=(11.7, 6.2))
    image = ax.imshow(np.abs(amb), extent=[h_grid.min(), h_grid.max(), eta_grid.min(), eta_grid.max()], origin="
     lower", aspect="auto", cmap="viridis")
    ax.set_xlabel("log-depth shift $h$")
    ax.set_ylabel(r"Mellin-frequency shift $\eta$")
    ax.set_title(r"Magnitude of the Gamma-sech ambiguity function, $\alpha=3/2$")
    fig.colorbar(image, ax=ax, fraction=0.035, pad=0.025, label="magnitude")
    save_figure(fig, "06_ambiguity_magnitude")

    # 07 ambiguity phase
    fig, ax = plt.subplots(figsize=(11.7, 6.2))
    image = ax.imshow(np.angle(amb), extent=[h_grid.min(), h_grid.max(), eta_grid.min(), eta_grid.max()], origin="
     lower", aspect="auto", cmap="twilight", vmin=-np.pi, vmax=np.pi)
    ax.set_xlabel("log-depth shift $h$")
    ax.set_ylabel(r"Mellin-frequency shift $\eta$")
    ax.set_title("Phase of the zero-free Gamma-sech ambiguity function")
    fig.colorbar(image, ax=ax, fraction=0.035, pad=0.025, label="phase")
    save_figure(fig, "07_ambiguity_phase")

    # 08 slices
    fig, axes = plt.subplots(1, 2, figsize=(12.8, 4.8))
    for eta, color in zip((0.0, 0.75, 1.5, 2.5), COLORS):
        axes[0].plot(h, np.abs(gamma_sech_kernel(1.5, h, eta)), lw=2, color=color, label=rf"$\eta={eta}$")
    axes[0].set_xlabel("$h$")
    axes[0].set_ylabel("magnitude")
    axes[0].set_title("Depth slices")
    axes[0].grid(alpha=0.18)
    axes[0].legend()
    eta_line = np.linspace(-8, 8, 1000)
    for hh0, color in zip((0.0, 0.8, 1.6, 2.4), COLORS):
        axes[1].plot(eta_line, np.abs(gamma_sech_kernel(1.5, hh0, eta_line)), lw=2, color=color, label=rf"$h={hh0}$"
     )
    axes[1].set_xlabel(r"$\eta$")
    axes[1].set_ylabel("magnitude")
    axes[1].set_title("Frequency slices")
    axes[1].grid(alpha=0.18)
    axes[1].legend()
    fig.suptitle("The ambiguity magnitude factorizes into depth and frequency envelopes", fontsize=16, weight="bold"
     )
    fig.tight_layout(rect=[0, 0, 1, 0.91])
    save_figure(fig, "08_ambiguity_slices")

    # 09 Poschl-Teller residual and potential
    h2 = np.linspace(-7, 7, 1600)
    eta0 = 0.8
    nu = 1.5 + 1j * eta0
    potential = -nu * (nu + 1) / np.cosh(h2 / 2) ** 2
    fig, axes = plt.subplots(1, 2, figsize=(12.8, 4.8))
    axes[0].plot(h2, potential.real, color=BLUE, lw=2, label="real")
    axes[0].plot(h2, potential.imag, color=MAGENTA, lw=2, label="imaginary")
```

```
    axes[0].set_title("Complex Poschl-Teller potential")
    axes[0].set_xlabel("$h$")
    axes[0].grid(alpha=0.18)
    axes[0].legend()
    f = gamma_sech_kernel(1.5, h2, eta0)
    axes[1].plot(h2, f.real, color=TEAL, lw=2, label="real")
    axes[1].plot(h2, f.imag, color=ORANGE, lw=2, label="imaginary")
    axes[1].set_title("Exact Gamma-sech eigenfunction")
    axes[1].set_xlabel("$h$")
    axes[1].grid(alpha=0.18)
    axes[1].legend()
    fig.suptitle("The generated ambiguity function solves a shape-invariant Schrodinger equation", fontsize=15.5,
     weight="bold")
    fig.tight_layout(rect=[0, 0, 1, 0.90])
    save_figure(fig, "09_poschl_teller")

    # 10 Wigner function
    y_w = np.linspace(-4.5, 3.0, 85)
    o_w = np.linspace(-5.5, 5.5, 95)
    wigner = wigner_log_gamma(1.5, y_w, o_w)
    fig, ax = plt.subplots(figsize=(11.7, 6.2))
    vmax = float(np.percentile(np.abs(wigner), 99.0))
    image = ax.imshow(wigner.T, extent=[y_w.min(), y_w.max(), o_w.min(), o_w.max()], origin="lower", aspect="auto",
     cmap="coolwarm", vmin=-vmax, vmax=vmax)
    ax.set_xlabel("log coordinate $y$")
    ax.set_ylabel(r"Mellin frequency $\omega$")
    ax.set_title(r"Wigner-Kontorovich-Lebedev packet, $\alpha=3/2$")
    fig.colorbar(image, ax=ax, fraction=0.035, pad=0.025, label="Wigner density")
    save_figure(fig, "10_wigner_kl")

    # 11 Wigner marginals
    fig, axes = plt.subplots(1, 2, figsize=(12.8, 4.8))
    axes[0].plot(y_w, np.trapezoid(wigner, o_w, axis=1), color=BLUE, lw=2.2, label="Wigner marginal")
    axes[0].plot(y_w, log_gamma_density(1.5, y_w), color=ORANGE, lw=1.5, ls="--", label="exact position law")
    axes[0].set_xlabel("$y$")
    axes[0].set_title("Position marginal")
    axes[0].grid(alpha=0.18)
    axes[0].legend()
    axes[1].plot(o_w, np.trapezoid(wigner, y_w, axis=0), color=TEAL, lw=2.2, label="Wigner marginal")
    axes[1].plot(o_w, meixner_density(1.5, o_w), color=MAGENTA, lw=1.5, ls="--", label="exact spectral law")
    axes[1].set_xlabel(r"$\omega$")
    axes[1].set_title("Frequency marginal")
    axes[1].grid(alpha=0.18)
    axes[1].legend()
    fig.suptitle("The Wigner packet recovers both exact probability marginals", fontsize=15.5, weight="bold")
    fig.tight_layout(rect=[0, 0, 1, 0.90])
    save_figure(fig, "11_wigner_marginals")

    # 12 uncertainty product
    alphas = np.logspace(-1, 2, 500)
    product = np.array([uncertainty_data(float(a))["product"] for a in alphas])
    fig, ax = plt.subplots(figsize=(12.6, 5.2))
    ax.semilogx(alphas, product, color=BLUE, lw=2.2)
    ax.axhline(0.5, color=RED, ls="--", lw=1.6, label="Heisenberg limit")
    ax.set_xlabel(r"shape parameter $\alpha$")
    ax.set_ylabel(r"$\Delta Y\,\Delta P$")
    ax.set_title(r"The log-Gamma packet approaches minimum uncertainty as $\alpha$ grows")
    ax.grid(alpha=0.18, which="both")
    ax.legend()
    save_figure(fig, "12_uncertainty_product")

    # 13 QFI geometry
    fig, ax = diagram_axes((12.6, 6.0))
    ax.text(0.5, 0.92, "Constant quantum geometry of the packet orbit", ha="center", fontsize=19, weight="bold")
    box(ax, (0.08, 0.58), 0.24, 0.18, "translation metric\n" + r"$g_{uu}=\alpha/4$", face=BLUE + "20", edge=BLUE,
     fontsize=13)
    box(ax, (0.38, 0.58), 0.24, 0.18, "modulation metric\n" + r"$g_{\eta\eta}=\psi_1(\alpha)$", face=TEAL + "20",
     edge=TEAL, fontsize=13)
    box(ax, (0.68, 0.58), 0.24, 0.18, "Berry curvature\n" + r"$\mathcal{F}_{u\eta}=1$", face=MAGENTA + "20", edge=
     MAGENTA, fontsize=13)
    box(ax, (0.22, 0.20), 0.23, 0.14, "QFI\n" + r"$\operatorname{diag}(\alpha,4\psi_1)$", face=WHITE, edge=PURPLE,
```

```
    fontsize=12)
    box(ax, (0.55, 0.20), 0.23, 0.14, "uncertainty\n" + r"$\alpha\psi_1(\alpha)>1$", face=WHITE, edge=ORANGE,
    fontsize=12)
    arrow(ax, (0.20, 0.58), (0.33, 0.34), color=PURPLE)
    arrow(ax, (0.50, 0.58), (0.33, 0.34), color=PURPLE)
    arrow(ax, (0.80, 0.58), (0.67, 0.34), color=ORANGE)
    ax.text(0.5, 0.08, "The metric is flat in packet coordinates while the symplectic curvature remains nonzero.",
    ha="center", fontsize=11.5, color=MUTED)
    save_figure(fig, "13_quantum_geometry")

    # 14 Laguerre packets
    y_l = np.linspace(-7, 4, 1800)
    fig, ax = plt.subplots(figsize=(12.6, 5.4))
    for ndeg, color in zip(range(6), COLORS):
        ax.plot(y_l, laguerre_packet(1.5, ndeg, y_l), lw=1.8, color=color, label=rf"$n={ndeg}$")
    ax.set_xlabel("log coordinate $y$")
    ax.set_ylabel(r"$\Lambda_{n,\alpha}(y)$")
    ax.set_title("Laguerre packet basis on logarithmic scale")
    ax.grid(alpha=0.18)
    ax.legend(ncol=6)
    save_figure(fig, "14_laguerre_packets")

    # 15 Meixner polynomials
    omega_m = np.linspace(-5.5, 5.5, 1600)
    rho_m = meixner_density(1.5, omega_m)
    fig, ax = plt.subplots(figsize=(12.6, 5.4))
    scale = np.sqrt(rho_m)
    for ndeg, color in zip(range(6), COLORS):
        ax.plot(omega_m, meixner_pollaczek_orthonormal(1.5, ndeg, omega_m) * scale, lw=1.8, color=color, label=rf"$n
    ={ndeg}$")
    ax.set_xlabel(r"Mellin frequency $\omega$")
    ax.set_ylabel(r"$p_n(\omega)\sqrt{\rho_\alpha(\omega)}$")
    ax.set_title("Meixner-Pollaczek packet basis in frequency coordinates")
    ax.grid(alpha=0.18)
    ax.legend(ncol=6)
    save_figure(fig, "15_meixner_packets")

    # 16 Fourier duality
    fig, axes = plt.subplots(2, 2, figsize=(12.8, 7.2), sharex=True)
    omega_d = np.linspace(-5, 5, 500)
    for ax, ndeg in zip(axes.ravel(), range(4)):
        exact = laguerre_packet_fourier(1.5, ndeg, omega_d)
        ax.plot(omega_d, exact.real, color=BLUE, lw=2, label="real")
        ax.plot(omega_d, exact.imag, color=MAGENTA, lw=2, label="imaginary")
        ax.set_title(rf"degree $n={ndeg}$")
        ax.grid(alpha=0.16)
    axes[0, 0].legend(ncol=2)
    for ax in axes[-1]:
        ax.set_xlabel(r"$\omega$")
    fig.suptitle("Fourier transform of Laguerre packets equals the Gamma envelope times Meixner-Pollaczek packets",
    fontsize=15.5, weight="bold")
    fig.tight_layout(rect=[0, 0, 1, 0.92])
    save_figure(fig, "16_fourier_duality")

    # 17 Laguerre Jacobi matrix
    jl = laguerre_jacobi(1.5, 12)
    fig, ax = plt.subplots(figsize=(8.4, 6.5))
    image = ax.imshow(jl, cmap="coolwarm", aspect="equal")
    ax.set_title("Laguerre multiplication operator")
    ax.set_xlabel("basis index")
    ax.set_ylabel("basis index")
    for i in range(12):
        for j in range(12):
            if abs(jl[i, j]) > 1e-12:
                ax.text(j, i, f"{jl[i,j]:.1f}", ha="center", va="center", fontsize=7, color=WHITE if abs(jl[i,j]) >
    4 else INK)
    fig.colorbar(image, ax=ax, fraction=0.045, pad=0.03)
    save_figure(fig, "17_laguerre_jacobi")

    # 18 Meixner Jacobi matrix
    jm = meixner_jacobi(1.5, 12)
```

```
    fig, ax = plt.subplots(figsize=(8.4, 6.5))
    image = ax.imshow(jm, cmap="coolwarm", aspect="equal")
    ax.set_title("Meixner-Pollaczek frequency operator")
    ax.set_xlabel("basis index")
    ax.set_ylabel("basis index")
    for i in range(12):
        for j in range(12):
            if abs(jm[i, j]) > 1e-12:
                ax.text(j, i, f"{jm[i,j]:.1f}", ha="center", va="center", fontsize=7, color=WHITE if abs(jm[i,j]) > 2.5 else INK)
    fig.colorbar(image, ax=ax, fraction=0.045, pad=0.03)
    save_figure(fig, "18_meixner_jacobi")

    # 19 SU(1,1) ladder
    fig, ax = diagram_axes((12.6, 6.2))
    ax.text(0.5, 0.93, r"One $\mathfrak{su}(1,1)$ ladder, two orthogonal coordinate systems", ha="center", fontsize=19, weight="bold")
    xs = np.linspace(0.08, 0.92, 7)
    for ndeg, xpos in enumerate(xs):
        box(ax, (xpos - 0.045, 0.58), 0.09, 0.11, rf"$|{ndeg}\rangle$", face=BLUE + "18", edge=BLUE, fontsize=12)
        if ndeg < len(xs) - 1:
            arrow(ax, (xpos + 0.045, 0.635), (xs[ndeg + 1] - 0.045, 0.635), color=TEAL, text=rf"$\sqrt{{({ndeg+1})({ndeg}+\alpha)}}$", text_offset=(0, 0.055))
    box(ax, (0.15, 0.20), 0.27, 0.15, "Laguerre coordinate\n" + r"$J_L=2K_0-K_+-K_-$", face=WHITE, edge=ORANGE, fontsize=12)
    box(ax, (0.58, 0.20), 0.27, 0.15, "Meixner coordinate\n" + r"$J_M=(K_++K_-)/2$", face=WHITE, edge=MAGENTA, fontsize=12)
    arrow(ax, (0.38, 0.58), (0.285, 0.35), color=ORANGE)
    arrow(ax, (0.62, 0.58), (0.715, 0.35), color=MAGENTA)
    ax.text(0.5, 0.08, r"The Casimir is $\frac{\alpha}{2}(\frac{\alpha}{2}-1)I$ in both atlases.", ha="center", fontsize=12, color=MUTED)
    save_figure(fig, "19_su11_duality")

    # 20 arithmetic convergence
    rows = summary["arithmetic_convergence"]
    scales = np.array([row["scale"] for row in rows], dtype=float)
    errors = np.array([row["error"] for row in rows], dtype=float)
    fig, ax = plt.subplots(figsize=(12.6, 5.2))
    ax.loglog(scales, errors, marker="o", color=BLUE, lw=2.2, label="measured kernel error")
    reference = errors[0] * (scales / scales[0]) ** (-0.25)
    ax.loglog(scales, reference, color=ORANGE, ls="--", lw=1.8, label=r"reference slope $m^{-1/4}$")
    ax.set_xlabel("base depth")
    ax.set_ylabel("complex kernel error")
    ax.set_title("Squarefree arithmetic packets converge to the Gamma-sech phase-space kernel")
    ax.grid(alpha=0.18, which="both")
    ax.legend()
    save_figure(fig, "20_arithmetic_convergence")

    # 21 complex trajectory comparison
    arithmetic = build_arithmetic_kernel(nmax=600_000)
    taus = np.linspace(-5, 5, 320)
    k0, l0 = 5000.0, 20000.0
    discrete = np.array([arithmetic.normalized_centered(k0, l0, float(t)) for t in taus])
    universal = np.array([universal_arithmetic_limit(arithmetic.alpha, arithmetic.beta, k0, l0, float(t)) for t in taus])
    fig, ax = plt.subplots(figsize=(7.2, 6.4))
    ax.plot(universal.real, universal.imag, color=ORANGE, lw=2.4, label="universal")
    ax.plot(discrete.real, discrete.imag, color=BLUE, lw=1.5, ls="--", label="squarefree arithmetic")
    ax.scatter([discrete[0].real, discrete[-1].real], [discrete[0].imag, discrete[-1].imag], color=[TEAL, MAGENTA], s=35)
    ax.set_xlabel("real part")
    ax.set_ylabel("imaginary part")
    ax.set_title("Centered depth-time trajectory")
    ax.grid(alpha=0.18)
    ax.legend()
    ax.set_aspect("equal", adjustable="box")
    save_figure(fig, "21_arithmetic_trajectory")

    # 22 fractional depth flow
    r_values = np.linspace(0.1, 8, 160)
    direct = np.array([fractional_mobius_transfer(float(r), nmax=120_000) for r in r_values])
```

```
    fig, ax = plt.subplots(figsize=(12.6, 5.2))
    ax.plot(r_values, direct, color=BLUE, lw=2.2)
    for integer in range(1, 9):
        ax.scatter([integer], [fractional_mobius_transfer(float(integer), nmax=120_000)], color=ORANGE, s=28)
    ax.set_xlabel("continuous depth $r$")
    ax.set_ylabel(r"$C_{2,2}(r,0)$")
    ax.set_title("The weighted finite-difference orbit extends to continuous depth")
    ax.grid(alpha=0.18)
    save_figure(fig, "22_fractional_depth")

    # 23 fractional binomial convergence
    terms = np.arange(4, 121, 4)
    target = float(fractional_binomial_transfer(2.5, terms=5000))
    approximations = []
    for count in terms:
        approximations.append(float(fractional_binomial_transfer(2.5, terms=int(count))))
    errors_frac = np.abs(np.array(approximations) - target)
    fig, ax = plt.subplots(figsize=(12.6, 5.2))
    ax.semilogy(terms, errors_frac, marker="o", color=TEAL, lw=2)
    ax.set_xlabel("binomial-series terms")
    ax.set_ylabel("absolute error")
    ax.set_title("Fractional-depth binomial reconstruction")
    ax.grid(alpha=0.18)
    save_figure(fig, "23_fractional_binomial")

    # 24 coherent-state lattice
    fig, ax = plt.subplots(figsize=(11.7, 6.1))
    uvals = np.linspace(-2.5, 2.5, 9)
    etavals = np.linspace(-4, 4, 9)
    for u in uvals:
        for eta in etavals:
            size = 30 + 220 * abs(complex(gamma_sech_kernel(1.5, u, eta)))
            ax.scatter(u, eta, s=size, facecolor=BLUE, edgecolor=INK, alpha=0.45)
    ax.set_xlabel("log-depth displacement $h$")
    ax.set_ylabel(r"Mellin-frequency displacement $\eta$")
    ax.set_title("A packet lattice weighted by the exact coherent-state overlap")
    ax.grid(alpha=0.15)
    save_figure(fig, "24_coherent_state_lattice")

    # 25 quantum architecture
    fig, ax = diagram_axes((12.6, 6.5))
    ax.text(0.5, 0.94, "Quantum and signal realization of the universal packet", ha="center", fontsize=19, weight="
    bold")
    box(ax, (0.04, 0.57), 0.18, 0.17, "number basis\n" + r"$|n\rangle$", face=BLUE + "20", edge=BLUE, fontsize=13)
    box(ax, (0.29, 0.57), 0.18, 0.17, "depth filter\n" + r"$e^{-kK_\beta}$", face=TEAL + "20", edge=TEAL, fontsize
    =13)
    box(ax, (0.54, 0.57), 0.18, 0.17, "logarithmic phase\n" + r"$e^{-itH}$", face=ORANGE + "20", edge=ORANGE,
    fontsize=13)
    box(ax, (0.79, 0.57), 0.17, 0.17, "packet overlap\n" + r"$\mathfrak{A}_\alpha$", face=MAGENTA + "20", edge=
    MAGENTA, fontsize=13)
    for start, end in [((0.22, 0.655), (0.29, 0.655)), ((0.47, 0.655), (0.54, 0.655)), ((0.72, 0.655), (0.79, 0.655)
    )]:
        arrow(ax, start, end)
    box(ax, (0.18, 0.20), 0.25, 0.15, "log-coordinate Gabor packet\ntranslation + modulation", face=WHITE, edge=
    PURPLE, fontsize=11.5)
    box(ax, (0.57, 0.20), 0.25, 0.15, "dual polynomial packets\nLaguerre + Meixner", face=WHITE, edge=RED, fontsize
    =11.5)
    arrow(ax, (0.38, 0.57), (0.305, 0.35), color=PURPLE)
    arrow(ax, (0.68, 0.57), (0.695, 0.35), color=RED)
    ax.text(0.5, 0.08, "The logarithmic Hamiltonian supplies the time coordinate. The calculus supplies the depth
    filter and packet geometry.", ha="center", fontsize=11.2, color=MUTED)
    save_figure(fig, "25_quantum_packet_realization")

    # 26 family map
    fig, ax = diagram_axes((12.8, 6.7))
    ax.text(0.5, 0.95, "Special Functions V: generated function family", ha="center", fontsize=20, weight="bold")
    nodes = [
        (0.06, 0.68, "Gamma-sech\nambiguity", BLUE),
        (0.30, 0.68, "log-Gamma\nwindow", TEAL),
        (0.54, 0.68, "Meixner\nspectral law", ORANGE),
        (0.78, 0.68, "Wigner-KL\nfunction", MAGENTA),
```

```
        (0.18, 0.28, "Laguerre-Meixner\nFourier pair", PURPLE),
        (0.46, 0.28, r"$\mathfrak{su}(1,1)$" + "\noperator pair", RED),
        (0.74, 0.28, "fractional-depth\ntransfer", YELLOW),
    ]
    for x0, y0, text, color in nodes:
        box(ax, (x0, y0), 0.17, 0.13, text, face=color + "18", edge=color, fontsize=11.2)
    for start, end in [
        ((0.23, 0.745), (0.30, 0.745)),
        ((0.47, 0.745), (0.54, 0.745)),
        ((0.71, 0.745), (0.78, 0.745)),
        ((0.385, 0.68), (0.265, 0.41)),
        ((0.625, 0.68), (0.545, 0.41)),
        ((0.865, 0.68), (0.825, 0.41)),
    ]:
        arrow(ax, start, end, color=INK)
    ax.text(0.5, 0.10, "All seven objects arise from one universal large-depth packet and its exact coordinate
     transforms.", ha="center", fontsize=11.5, color=MUTED)
    save_figure(fig, "26_function_family_map")


# ---------------------------------------------------------------------------
# Tables and output ledger
# ---------------------------------------------------------------------------


def write_tables(summary: dict[str, object], checks: list[dict[str, object]]) -> None:
    uncertainty_rows = []
    for alpha in (0.5, 0.75, 1.0, 1.5, 2.0, 3.0, 5.0, 10.0):
        row = {"alpha": alpha, **uncertainty_data(alpha)}
        uncertainty_rows.append(row)
    with (TABLES / "uncertainty.csv").open("w", newline="") as handle:
        writer = csv.DictWriter(handle, fieldnames=list(uncertainty_rows[0].keys()))
        writer.writeheader()
        writer.writerows(uncertainty_rows)

    with (TABLES / "arithmetic_convergence.csv").open("w", newline="") as handle:
        rows = summary["arithmetic_convergence"]
        writer = csv.DictWriter(handle, fieldnames=list(rows[0].keys()))
        writer.writeheader()
        writer.writerows(rows)

    with (RESULTS / "checks.csv").open("w", newline="") as handle:
        writer = csv.DictWriter(handle, fieldnames=["check", "passed", "detail"])
        writer.writeheader()
        writer.writerows(checks)

    headline = rf"""
\begin{{center}}
\begin{{tabular}}{{L{{0.39\textwidth}}L{{0.47\textwidth}}}}
\toprule
Certificate & Value\\
\midrule
Built-in checks & {summary['built_in_checks']['passed']} of {summary['built_in_checks']['total']} passed\\
Ambiguity formula error & ${summary['ambiguity_max_error']:.3e}$\\
Laguerre--Meixner Fourier error & ${summary['fourier_duality_max_error']:.3e}$\\
Meixner Gram error & ${summary['meixner_gram_error']:.3e}$\\
Laguerre Gram error & ${summary['laguerre_gram_error']:.3e}$\\
$\mathfrak{{su}}(1,1)$ commutator error & ${summary['su11_commutator_error']:.3e}$\\
Poschl--Teller residual & ${summary['poschl_teller_error']:.3e}$\\
Fractional-depth error & ${summary['fractional_depth_error']:.3e}$\\
\bottomrule
\end{{tabular}}
\end{{center}}
"""
    (GENERATED / "headline.tex").write_text(headline)

    lines = [
        r"\begin{longtable}{L{0.42\textwidth}L{0.12\textwidth}L{0.37\textwidth}}",
        r"\caption{Built-in verification ledger.}\label{tab:checks}\\",
        r"\toprule Check & Result & Detail\\\midrule",
        r"\endfirsthead",
```

```
        r"\toprule Check & Result & Detail\\\midrule",
        r"\endhead",
    ]
    for check in checks:
        name = str(check["check"]).replace("_", r"\_")
        detail = str(check["detail"]).replace("_", r"\_")
        result = "pass" if check["passed"] else "fail"
        lines.append(f"{name} & {result} & {detail}\\\\")
    lines += [r"\bottomrule", r"\end{longtable}"]
    (GENERATED / "checks.tex").write_text("\n".join(lines))

    uncertainty_tex = [
        r"\begin{center}",
        r"\begin{tabular}{rrrrr}",
        r"\toprule $\alpha$ & $\operatorname{Var}Y$ & $\operatorname{Var}P$ & $\Delta Y\Delta P$ & $\mathcal I_{\eta \eta}$\\\midrule",
    ]
    for row in uncertainty_rows:
        uncertainty_tex.append(
            f"{row['alpha']:.2f} & {row['var_y']:.6f} & {row['var_p']:.6f} & {row['product']:.6f} & {row['qfi_eta']:.6f}\\\\"
        )
    uncertainty_tex += [r"\bottomrule", r"\end{tabular}", r"\end{center}"]
    (GENERATED / "uncertainty.tex").write_text("\n".join(uncertainty_tex))

    convergence_tex = [
        r"\begin{center}",
        r"\begin{tabular}{rrrr}",
        r"\toprule base depth & kernel error & real part & imaginary part\\\midrule",
    ]
    for row in summary["arithmetic_convergence"]:
        convergence_tex.append(
            f"{row['scale']} & {row['error']:.6e} & {row['discrete_real']:.8f} & {row['discrete_imag']:.8f}\\\\"
        )
    convergence_tex += [r"\bottomrule", r"\end{tabular}", r"\end{center}"]
    (GENERATED / "arithmetic_convergence.tex").write_text("\n".join(convergence_tex))


def main() -> None:
    checks, summary = perform_checks()
    generate_figures(summary)
    write_tables(summary, checks)
    summary["figure_pairs"] = len(list(FIGURES.glob("*.pdf")))
    (RESULTS / "results.json").write_text(json.dumps(summary, indent=2))
    print(json.dumps(summary["built_in_checks"], indent=2))


if __name__ == "__main__":
    main()
```

Appendix B

Reproduction

From the source directory, run

```
python projector_generated_special_functions_v.py
pytest -q
pdflatex -interaction=nonstopmode Projector_Generated_Special_Functions_V.tex
pdflatex -interaction=nonstopmode Projector_Generated_Special_Functions_V.tex
pdflatex -interaction=nonstopmode Projector_Generated_Special_Functions_V.tex
```

The generator writes all figures and tables before the manuscript is compiled.